Ejercicio Práctico · Arquitectura de Integración

ARQUITECTURA DE INTEGRACION PARA MODERNIZACION BANCARIA

Banco DeMuestra

Por Stanley Llaguno Caicedo

Contenido

1. Resumen Ejecutivo 5
2. Principios Arquitectonicos 5
Estandar Bian 5
API-First y Contratos Explícitos 5
Event-Driven Architecture 5
Zero Trust Security 6
Strangler Fig para Migración sin Riesgo 6
Stack de Frontend — Decisión Tecnológica 6
Polyglot Persistence 6
Clean Architecture + SOLID en cada Microservicio 6
Mapeo Explícito — SOLID en Servicios, APIs, Eventos y Adaptadores 7
3. Diagramas C4 9
Nivel 1 — Contexto del Sistema 9
Nivel 2 – Contenedores 10
Nivel 3 — Componentes: Payment Service 11
Nivel 3 — Componentes: Account Service 11
Flujos Críticos — Diagramas de Secuencia 13
Onboarding de Cliente — KYC y CDD 15
Detección de Fraude en Tiempo Real 17
4. Patrones de integracion 19
Decisión de Diseño: Llamadas Síncronas vs Asíncronas en el Critical Path de Pago 23
Controles que Previenen Cascada de Fallos en las Llamadas Síncronas 25
CQRS y Event Sourcing — Implementación Detallada 27
Event Sourcing — Estructura del Event Store 28
Ciclo de Vida de un Pago en Event Sourcing 29
Snapshot Strategy — Rendimiento a Escala 29
Proyectores — Read Models por Caso de Uso 30
Stream Versioning — Evolución de Eventos sin Breaking Changes 30
Reconstrucción de Proyector Caído 31
MLOps — Ciclo de Vida del Modelo de Detección de Fraude 31
Arquitectura MLOps — Componentes 31
Feature Store — Features de Detección de Fraude 32
Model Registry — Versionado y Linaje 33
Explicabilidad de Decisiones (Requisito Regulatorio) 34
Feedback Loop — Reentrenamiento Continuo 34
5. Estrategia de APIs Internas y Externas 35
APIs Internas (Inter-Servicio) 35
APIs Externas (Open Finance / Canales) 36
Ejemplos de APIs — Formato BIAN v14 (Release 14.0.0) 36

Consulta de Saldo — CurrentAccount / Retrieve (API Interna) 37
Iniciación de Pago — PaymentExecution / Initiate (API Interna) 38
Consulta de Estado de Pago — PaymentExecution / Retrieve (API Interna) 39
Autorización Open Finance — PaymentExecution / Execute (API Externa, FAPI 1.0).. 39
Alerta de Fraude — FraudDiagnosis / Notify (AsyncAPI — Evento) 40
6. Arquitectura de Eventos (Apache Kafka) 40
Definición de Canal AsyncAPI 3.x — Payment Execution / Notify 40
Ejemplo de Mensaje en Tiempo de Ejecución 43
Catálogo de Eventos Principales 44
Lineamientos de Calidad de Eventos 44
Integración SWIFT — ISO 20022 MX Messages 45
Familias de Mensajes ISO 20022 Utilizadas 45
Arquitectura de Conectividad SWIFT en Azure 46
Flujo Completo — Transferencia Internacional (pacs.008) 47
Campos Clave en MX vs MT Legacy 47
Manejo de Rechazos — R-Messages (pacs.002) 48
Arquitectura AML/CFT — Antilavado de Dinero y Financiamiento del Terrorismo 48
Tipologías de Detección — Rule Engine 49
SAR Pipeline — Suspicious Activity Report 50
CTR — Currency Transaction Report 50
Customer Due Diligence (CDD) y Niveles de Riesgo 50
Sanctions Screening — Listas de Verificación 51
7. Seguridad, IAM y Protección de Datos Personales 51
Flujo de Inyección de Credenciales — ACL Adapter Legacy 52
Controles de Seguridad por Capa 53
Protección de Datos Personales (LOPD / GDPR) 53
PCI DSS 4.0 — Cardholder Data Environment (CDE) 54
Datos en Alcance PCI DSS — CHD vs SAD 54
Estrategia de Tokenización — Reducción de Alcance CDE 55
Segmentación de Red del CDE en Azure 56
Gestión de Claves de Cifrado (PCI DSS Req. 3.5 / 3.6) 57
Controles PCI DSS 4.0 — Cumplimiento por Requisito 57
PAN Masking en Logs y APIs 58
8. Alta Disponibilidad y Recuperación ante Desastres 58
Alta Disponibilidad (Intra-Región) 58
Recuperación ante Desastres (Inter-Región) 59
Estrategia: Active-Passive con failover automático 59
9. Estrategia de Integración Multicore 59
Principios del Multicore 59

Flujo de Routing Multicore ... 60
Sincronización entre Cores ... 60
Conectividad con el Core Bancario Tradicional ... 60
Topología de Red — Datacenter On-Premise ↔ Azure ... 61
Azure ExpressRoute — Conectividad Dedicada ... 61
Protocolos Legacy Soportados por el ACL Adapter ... 62
Resiliencia de la Conexión Legacy ... 62
Monitoreo de la Conexión Legacy ... 63
10. Gobierno de APIs y Microservicios ... 63
Ciclo de Vida de APIs ... 63
Gobierno de Microservicios ... 63
11. Plan de Migración Gradual ... 63
12. Observabilidad y Trazabilidad ... 64
Los Tres Pilares ... 64
Correlation ID — Trazabilidad de Extremo a Extremo ... 65
Alertas y SLO Monitoring ... 65
Métricas Regulatorias Clave y su Fuente de Datos ... 66
Arquitectura del Regulatory Reporting Pipeline ... 67
Cálculo de RWA — Enfoque Estándar (SA) vs Modelos Internos (IRB) ... 68
Frecuencia de Cálculo y Reporte ... 68
Formato de Reporte — COREP / XBRL ... 69
Estrategia de Testing — Calidad y Seguridad en Todos los Niveles ... 69
Pirámide de Testing por Capa ... 69
Perfiles de Carga — Performance Testing (Gatling / k6) ... 70
Reconciliación Financiera — T+0, T+1 y Nostro Management ... 71
Reconciliación Intraday (T+0) — Posición en Tiempo Real ... 71
Reconciliación de Cierre (T+0 EOD — Close of Business) ... 71
Reconciliación T+1 — Settlement y Nostro ... 72
13. Pipeline CI/CD — DevSecOps ... 72
Herramientas y Responsabilidades ... 72
Stages del Pipeline — Flujo Completo ... 73
Open Finance — Framework Regulatorio y Diseño de Consentimiento ... 74
Elementos Comunes a Todos los Frameworks de Open Finance ... 74
Flujo de Consentimiento — FAPI 1.0 Advanced ... 75
Anexo A — Matriz de Dominios BIAN (Release v14.0.0) ... 76
A.2 Control Records y Behavior Qualifiers por Servicio ... 76
Anexo B — Matriz de Sistemas de Registro ... 78
Anexo C — Tabla de Controles de Seguridad ... 79

1. Resumen Ejecutivo

Banco DeMuestra(es el nombre que para este case esenario que vamos a utilizar) enfrenta el desafío de modernizar su infraestructura tecnológica sin interrumpir las operaciones del core bancario tradicional. Esta propuesta presenta una **arquitectura de integración event-driven híbrida** sobre Microsoft Azure, diseñada para conectar el sistema core legacy con un nuevo core bancario digital, exponer APIs bajo el estándar Open Finance (FAPI), y cumplir con los requisitos regulatorios y de protección de datos personales.

La solución se articula en torno a tres pilares:

1. **API Gateway centralizado (Azure APIM)** como único punto de entrada para canales digitales, socios Open Finance y comunicación inter-servicio.
2. **Event Bus (Apache Kafka / Azure Event Hubs)** como columna vertebral de la integración asíncrona, garantizando desacoplamiento, trazabilidad y consistencia eventual entre todos los sistemas.
3. **Microservicios alineados al estándar BIAN**, con responsabilidades únicas, contratos explícitos vía OpenAPI, y bases de datos propias (polyglot persistence).

Latencia Intra-Region	SLA Canales Digitales	Recovery Point Objective	Recovery Time Objective	Plan de Migracion
-15ms	**99%**	**15'**	**RTO 30'**	**24m**

Decision de la nube: Microsoft Azure fue seleccionado por su ecosistema de cumplimiento bancario en LATAM (ISO 27001, PCI DSS, SOC 1/2, FIPS 140-2), su oferta de servicios financieros especializados (Azure APIM, Event Hubs con API Kafka, AKS, AD B2C) y sus regiones disponibles en la región (Brazil South, Chile Central, Mexico Central), que garantizan baja latencia y soberanía de datos.

2. Principios Arquitectonicos

Estandar Bian

El *Banking Industry Architecture Network* (BIAN) define Service Domains independientes con responsabilidades acotadas. Cada microservicio de esta solución está alineado a un Service Domain BIAN, lo que asegura semántica compartida, evolución independiente y portabilidad entre proveedores.

API-First y Contratos Explícitos

Todo componente expone su contrato mediante **OpenAPI 3.0** antes de ser codificado. Los contratos son la fuente de verdad, versionados en repositorio Git, y validados con *consumer-driven contract testing* (Pact). Las APIs externas cumplen el perfil **FAPI 1.0 Advanced** para Open Finance.

Event-Driven Architecture

La comunicación entre dominios es preferentemente asíncrona vía eventos en Kafka. Esto garantiza bajo acoplamiento, resiliencia ante fallos parciales, y un historial auditale completo de todas las operaciones bancarias. Los servicios son productores o consumidores, nunca dependientes directos entre sí.

Zero Trust Security

Ningún componente es confiable por defecto. Todo tráfico — interno y externo — está autenticado (mTLS entre servicios, OAuth 2.0 + OIDC para usuarios), autorizado (RBAC/ABAC por operación), y cifrado en tránsito (TLS 1.3).

Strangler Fig para Migración sin Riesgo

El API Gateway actúa como fachada frente al sistema legacy. El tráfico se migra gradualmente hacia los nuevos microservicios mediante *feature flags* y reglas de routing, sin *big-bang migrations* ni ventanas de mantenimiento extendidas.

Stack de Frontend — Decisión Tecnológica

Para los canales digitales se adopta **Angular 17+ con TypeScript** como framework estándar para Web Banking y Portal Backoffice.

Y dentro de **Mobile App** usa **React Native** — estándar en neobanks (Nubank, Revolut, Monzo) — por su única codebase para iOS y Android, rendimiento nativo, y ecosistema maduro de componentes financieros.

Polyglot Persistence

Cada microservicio es propietario exclusivo de su base de datos. No se comparten bases de datos entre servicios. El tipo de base de datos se elige según el patrón de acceso: Azure SQL para transacciones ACID, Azure Cosmos DB para alta velocidad de lectura en tiempo real (fraudes), Redis para caché de sesiones e idempotencia.

Clean Architecture + SOLID en cada Microservicio

Cada microservicio está estructurado en capas (Domain → Application → Infrastructure → Interface), garantizando que la lógica de negocio no dependa de frameworks, bases de datos ni APIs externas. Se aplican los cinco principios SOLID:

- **SRP:** cada servicio tiene un único dominio de responsabilidad (ej. Payment Service solo orquesta pagos).
- **OCP:** nuevos canales de pago (SWIFT, ACH, SPEI) se añaden sin modificar el orquestador, solo extendiendo adaptadores.
- **LSP:** los adaptadores de core (legacy y digital) son intercambiables a través de la misma interfaz CoreBankingPort.
- **ISP:** los clients HTTP hacia otros servicios exponen solo las operaciones que necesitan, no interfaces generales.
- **DIP:** los microservicios dependen de abstracciones (ports), no de implementaciones concretas (adapters).

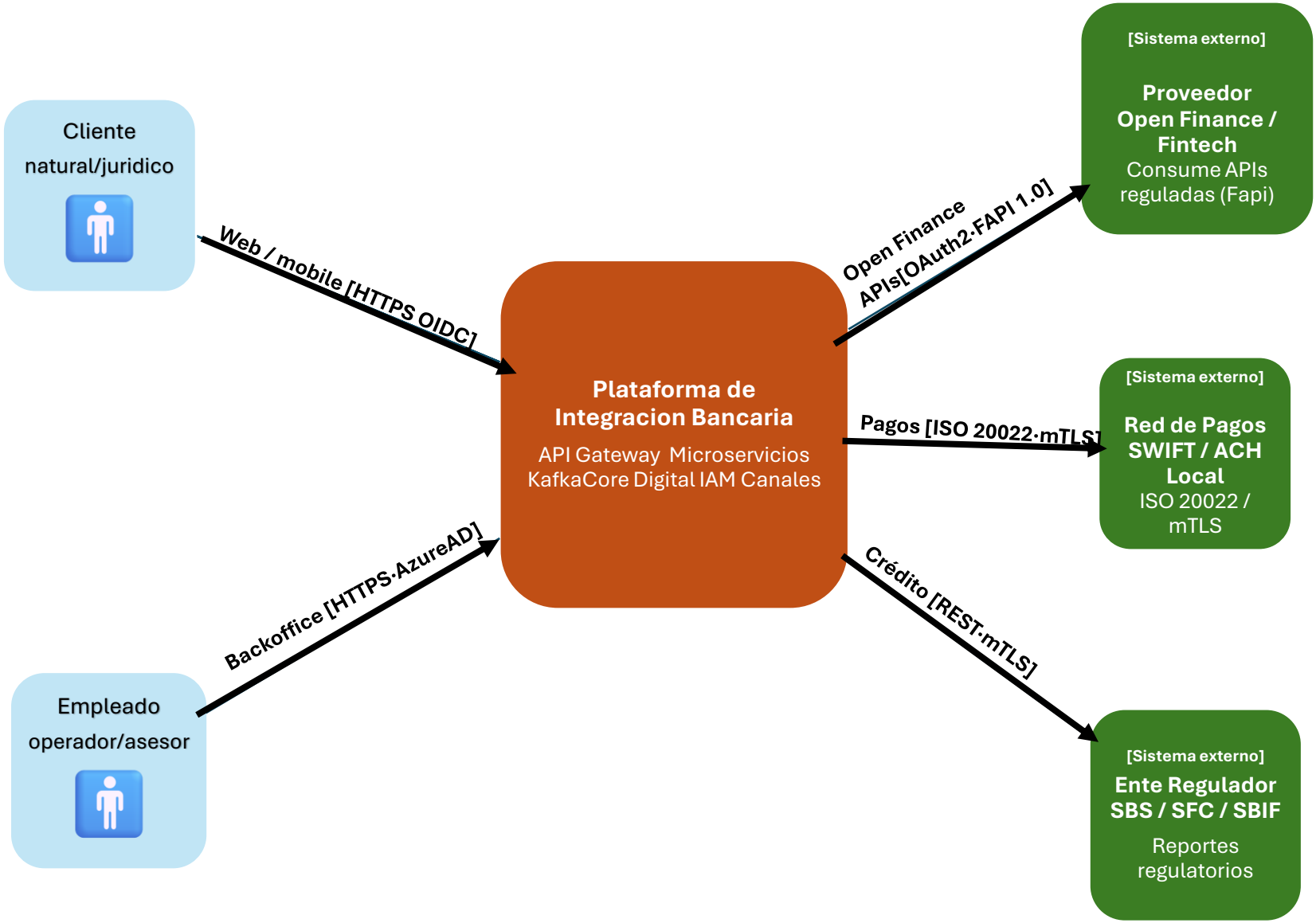
Mapeo Explícito — SOLID en Servicios, APIs, Eventos y Adaptadores

Principio	Manifestación en el diseño	Componente / Límite concreto	Violación que se evita
SRP Responsabilidad Única	Cada microservicio posee exactamente un dominio BIAN. El **Payment Service** solo orquesta pagos — no consulta saldos, no gestiona clientes, no detecta fraude. Cada una de esas responsabilidades vive en un servicio separado. Dentro de cada servicio: ***AccountCommandHandler*** solo deserializa y delega. ***AccountDomainService*** solo aplica reglas de negocio. ***OutboxRepository*** solo persiste**.** ***AccountEventPublisher*** solo publica. Ninguno hace más de una cosa.	Límites de servicio: Customer Svc / Account Svc / Payment Svc / Fraud Svc / Risk Svc cada uno con su propia DB, sin acceso cruzado. Componentes internos (C4 L3): ***CommandHandler*** ***DomainService*** ***CoreRouter*** ***OutboxRepository*** ***EventPublisher*** ***QueryHandler*** Cada uno con una única razón para cambiar	Un *God Service* que orquesta pagos, consulta saldos, valida fraude y notifica al cliente desde un único componente. Cualquier cambio en la lógica de fraude obligaría a redesplegar el servicio de pagos completo.
OCP Abierto/Cerrado	El **CoreRouter** está cerrado a modificación pero abierto a extensión: añadir un tercer proveedor de core bancario (ej. un BaaS cloud) requiere implementar un nuevo CoreBankingAdapter — ninguna línea del CoreRouter ni del Payment Service cambia. El **API Gateway (APIM)** aplica políticas como plugins — WAF, rate limiting, logging — sin modificar los microservicios. Añadir una nueva política no toca el código de ningún servicio. Los **Projectors** (CQRS) son extensión pura: añadir un nuevo read model = añadir un nuevo Projector que consume el mismo stream de eventos. El Event Store no se modifica.	***CoreBankingPort*** (interfaz): cerrada a modificación. ***LegacyCoreAdapter***, ***DigitalCoreAdapter***: extensiones cerradas entre sí. Event Store (append-only): nunca modificado, solo extendido con nuevos eventos. APIM policies: extensibles via XML policy expressions sin tocar microservicios.	Un switch(coreType) en el Payment Service que hay que modificar cada vez que se agrega un core. Un API Gateway con lógica de negocio embebida que crece y se vuelve inmantenible.
LSP Sustitución de Liskov	***LegacyCoreAdapter*** *y* ***DigitalCoreAdapter*** implementan la misma interfaz CoreBankingPort. El **Account Service** los usa de forma intercambiable — el comportamiento observable (débito exitoso, saldo actualizado, evento publicado) es idéntico independientemente de cuál esté activo. Durante la migración multicore (Sección 9), el Account Service opera con *ambos* adapters simultáneamente según el ***coreType*** de la cuenta. El servicio nunca necesita saber cuál está usando — es transparente.	Interfaz***: CoreBankingPort { getBalance(accountId), executeDebit(accountId, amount, correlationId), executeCredit(...) }*** Implementaciones***: LegacyCoreAdapter*** (IBM MQ) y ***DigitalCoreAdapter*** (REST). Ambas devuelven el mismo tipo de resultado y lanzan las mismas excepciones de dominio.	Un adapter que devuelve null en lugar de lanzar excepción en caso de error, o que tiene efectos secundarios adicionales (e.g., publicar un evento extra) que el contrato no define. Eso rompería el comportamiento esperado por el Account Service.
ISP Segregación de Interfaces	El **Account Service** expone dos puertos completamente separados: ***ICommandPort*** (Initiate, Execute, Control) e ***IQueryPort*** (Retrieve, Exchange). El componente de Read Side (***AccountQueryHandler***) implementa solo ***IQueryPort*** — nunca ve ni depende de los comandos. Los **clientes HTTP** entre microservicios (e.g., el cliente que Payment Service usa para llamar a Fraud Detection) exponen solo ***checkFraud***(***PaymentRequest***): ***FraudResult*** — no una interfaz genérica de Fraud Service con todos sus métodos internos. Los **scopes de Open Finance** son interfaces de autorización	***ICommandPort vs IQueryPort*** en Account Service (C4 L3, Sección 3.4). Clientes Feign/gRPC stub por operación: ***FraudCheckClient, RiskAssessmentClient*** — no un cliente genérico de cada servicio. OAuth 2.0 scopes en APIM Externo como interfaces de autorización granulares.	interfaz ***IAccountService*** con 20 métodos que obliga a todo componente que necesite solo consultar saldo a depender también de todos los métodos de escritura. Cambios en los métodos de escritura romperían los tests y el build de componentes que solo leen.

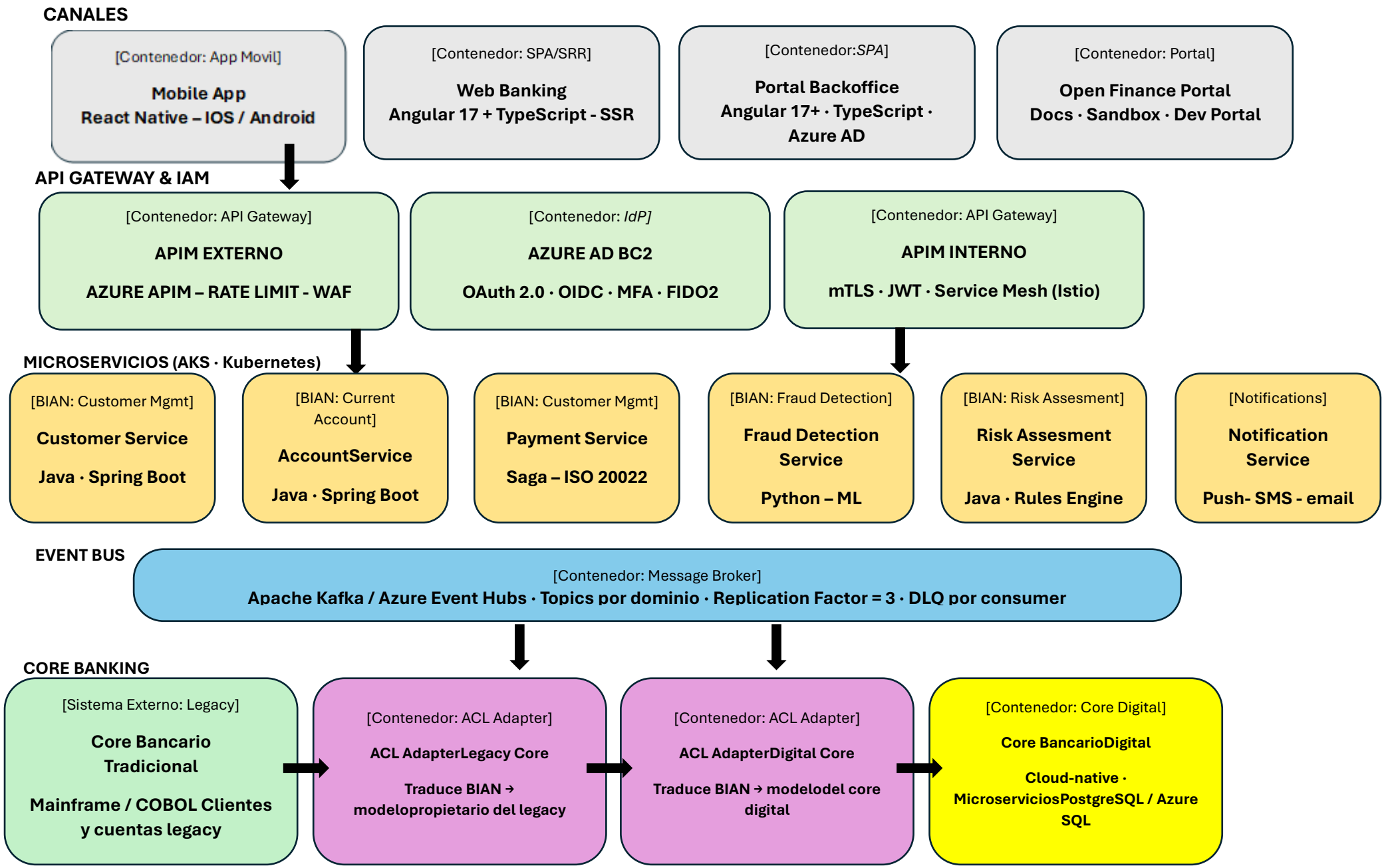
	segregadas: ***accounts***:***read***, ***transactions***:***read***, ***payments***:***write***. Un TPP con scope ***accounts***:***read*** no puede ver la interfaz de pagos.		
DIP Inversión de Dependencias	El **AccountDomainService** (capa Domain, la más interna de Clean Architecture) no importa ninguna clase de infraestructura. Depende de la interfaz ***CoreBankingPort*** — que está definida en el dominio. La implementación concreta (***LegacyCoreAdapter***, que sabe de IBM MQ y Azure) vive en la capa Infrastructure y depende del dominio, nunca al revés. El **Fraud Detection Service** consume features del Feature Store mediante una interfaz ***IFeatureStore***. La implementación (Redis para online features, Azure ML Feature Store para offline) es un detalle de infraestructura que el modelo ML nunca conoce. Los **eventos de Kafka** siguen el mismo principio: el Payment Service publica un evento de dominio (***PaymentAuthorized***) sin saber quién lo consume. El Notification Service, Account Service y Reporting Service dependen del contrato del evento — no de Payment Service directamente.	Clean Architecture: la regla de dependencia apunta siempre hacia adentro (Domain ← Application ← Infrastructure ← Interface). Nunca al revés. ***CoreBankingPort*** definida en capa Domain. Implementada en Infrastructure. Eventos Kafka como contratos de dominio (AsyncAPI 3.x en Sección 6): el productor y los consumidores dependen del schema del evento, no entre sí.	Un ***AccountDomainService*** que importa ***IBMMQClient*** directamente. Al cambiar de IBM MQ a otro broker, habría que modificar la lógica de negocio del dominio. Los unit tests del dominio requerirían un IBM MQ real o un mock frágil.

3. Diagramas C4

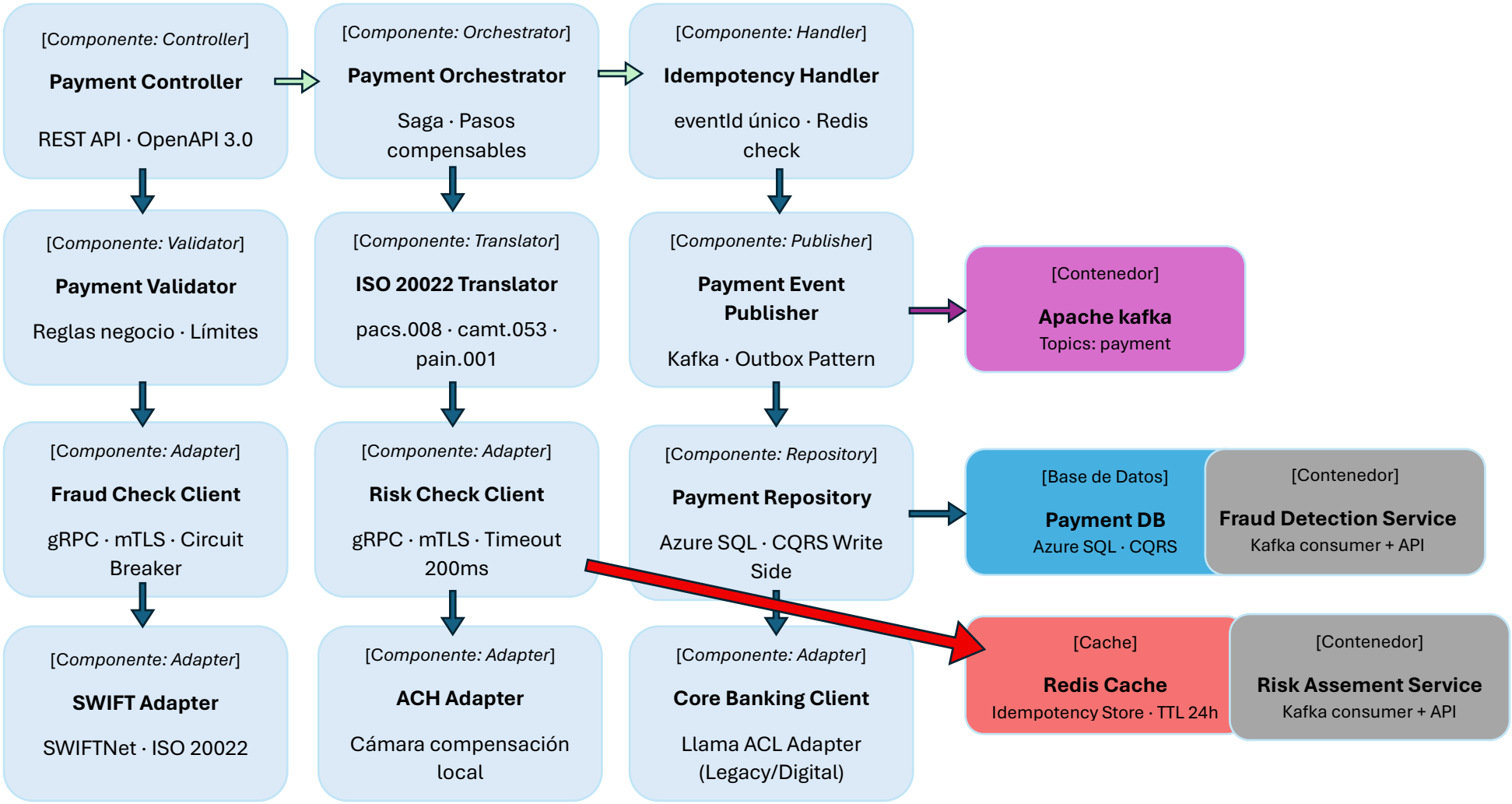
Nivel 1 — Contexto del Sistema



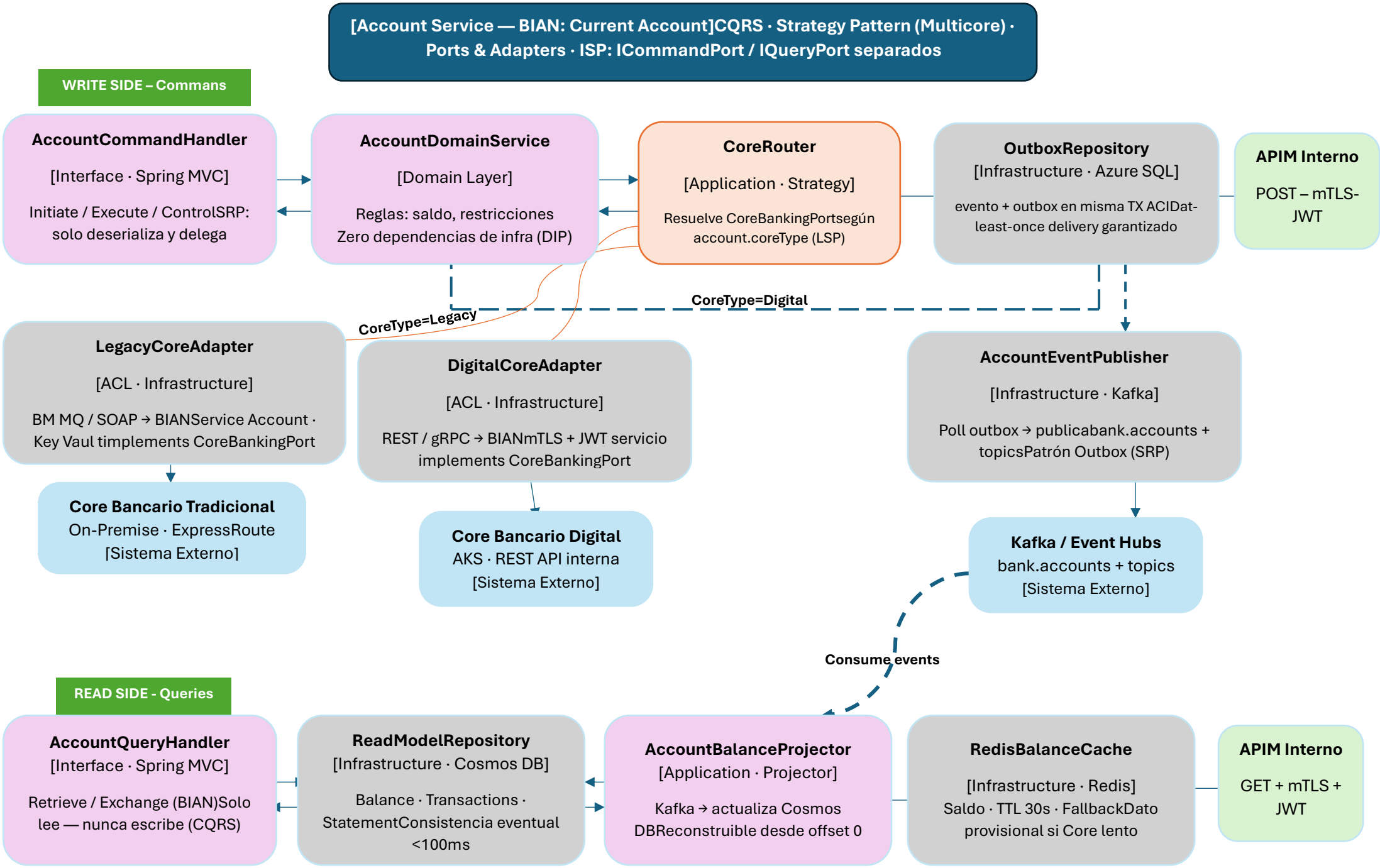
Nivel 2 – Contenedores



Nivel 3 — Componentes: Payment Service



Nivel 3 — Componentes: Account Service

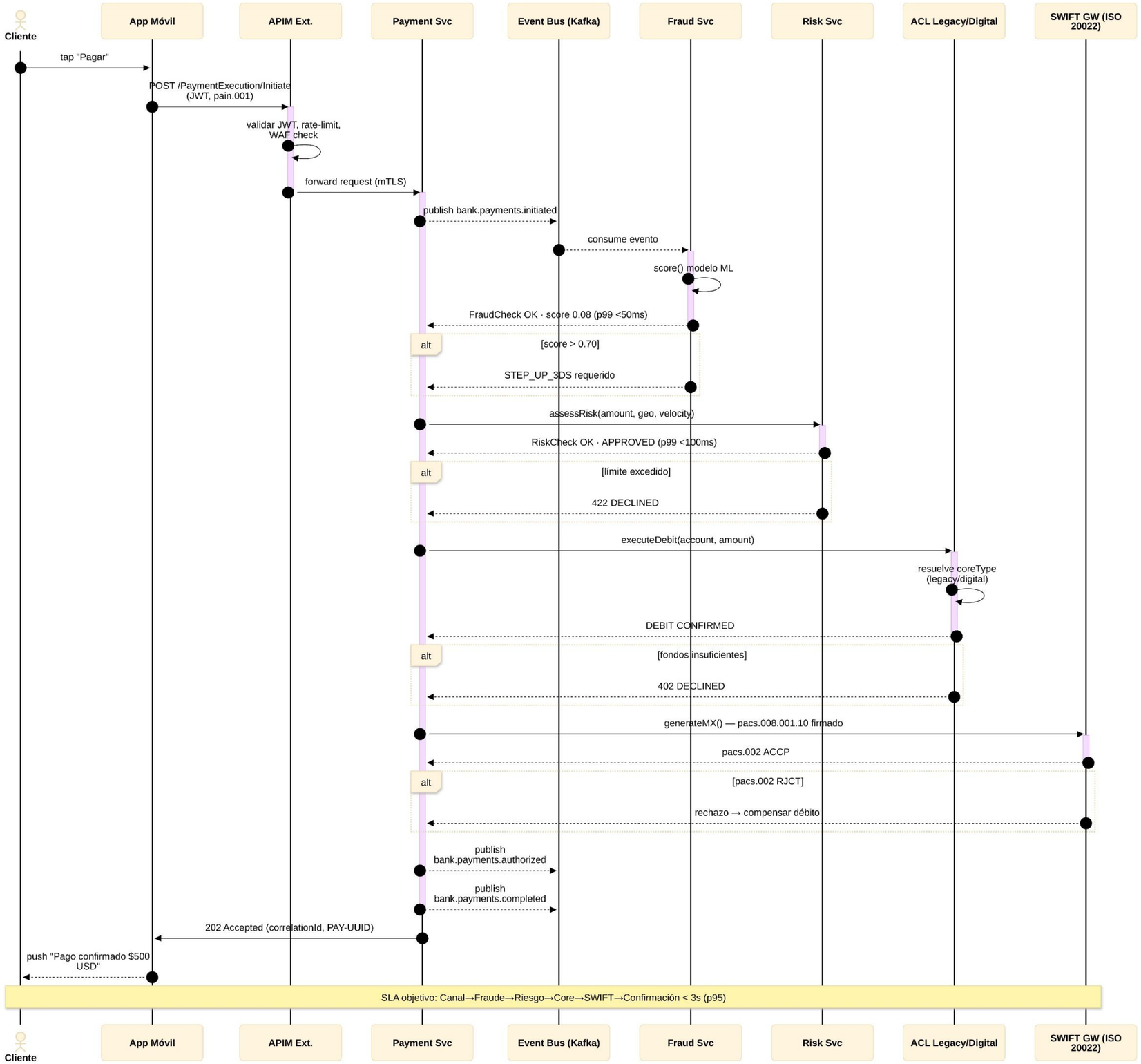


Flujos Críticos — Diagramas de Secuencia

Los diagramas de secuencia complementan los diagramas C4 (estructura estática) mostrando el comportamiento dinámico del sistema. Se presentan los tres flujos más críticos: autorización de pago, onboarding de cliente con KYC, y detección de fraude en tiempo real.

Tramo	Latencia objetivo p99	Protocolo
Mobile App → APIM Externo	<100ms	HTTPS / TLS 1.3
APIM → Payment Service	<20ms	HTTP/2 interno (mTLS)
Payment Service → Fraud Service	<50ms	gRPC (sync, en Critical Path)
Payment Service → Risk Service	<100ms	gRPC (sync, en Critical Path)
Payment Service → ACL (Core Legacy)	<500ms	IBM MQ / SOAP (ACL Adapter)
ISO 20022 Translation + SWIFT	<1500ms	pacs.008 XML / red SWIFT
Total end-to-end (p95)	**<3s**	—

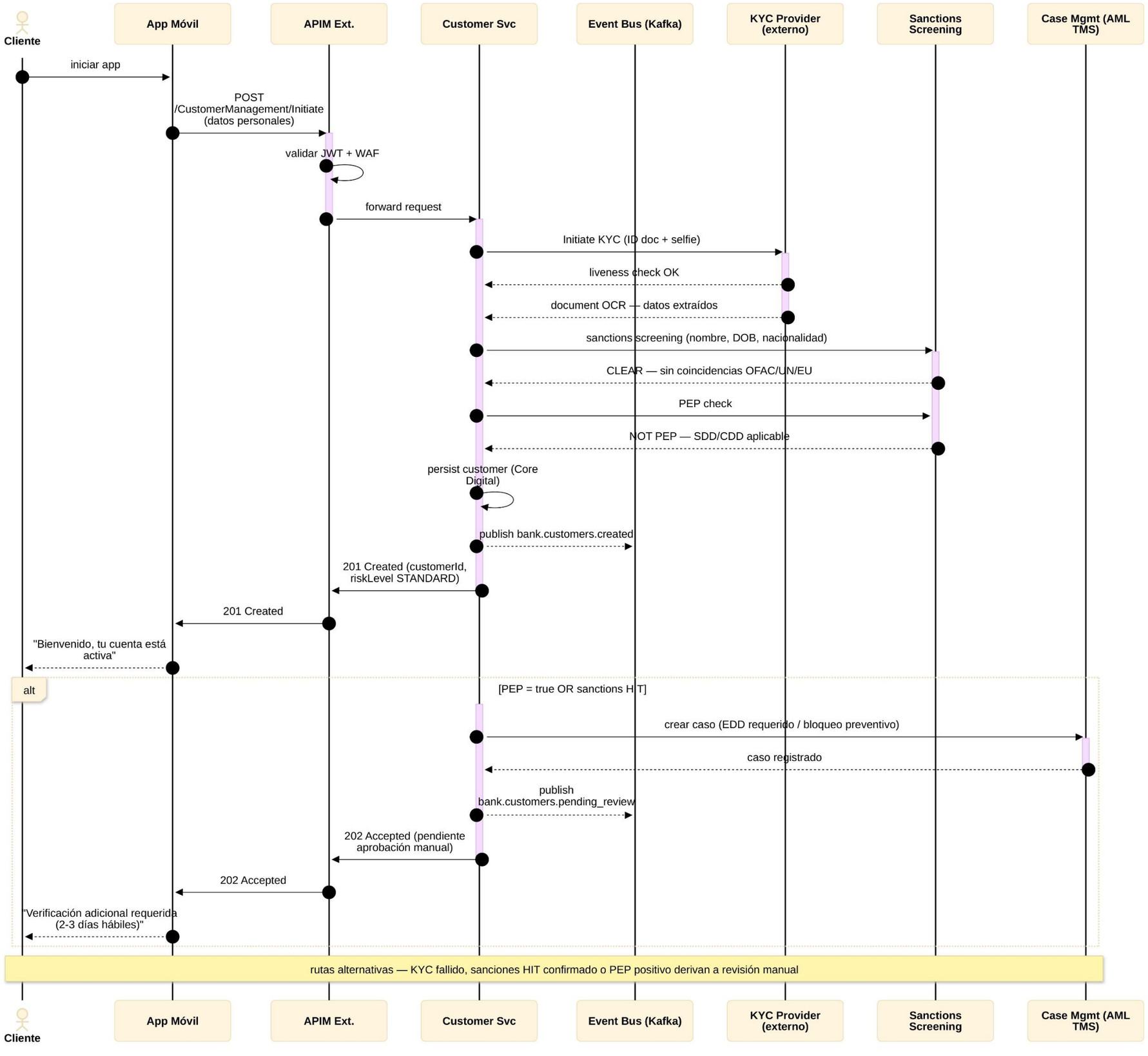
Autorización de Pago Internacional — Flujo Completo



Onboarding de Cliente — KYC y CDD

Paso	**SLA**	**Resultado posible**
Liveness check + OCR de documento	<10s	OK → continúa / FAIL → solicitar reintento (máx. 3)
Sanctions + PEP screening	<200ms	CLEAR → SDD/CDD / HIT → EDD + revisión manual
Aprobación automática (SDD/CDD)	Tiempo real	Cuenta activa inmediata
Aprobación manual (EDD / PEP)	2–5 días hábiles	Cuenta activa o rechazada + SAR si aplica

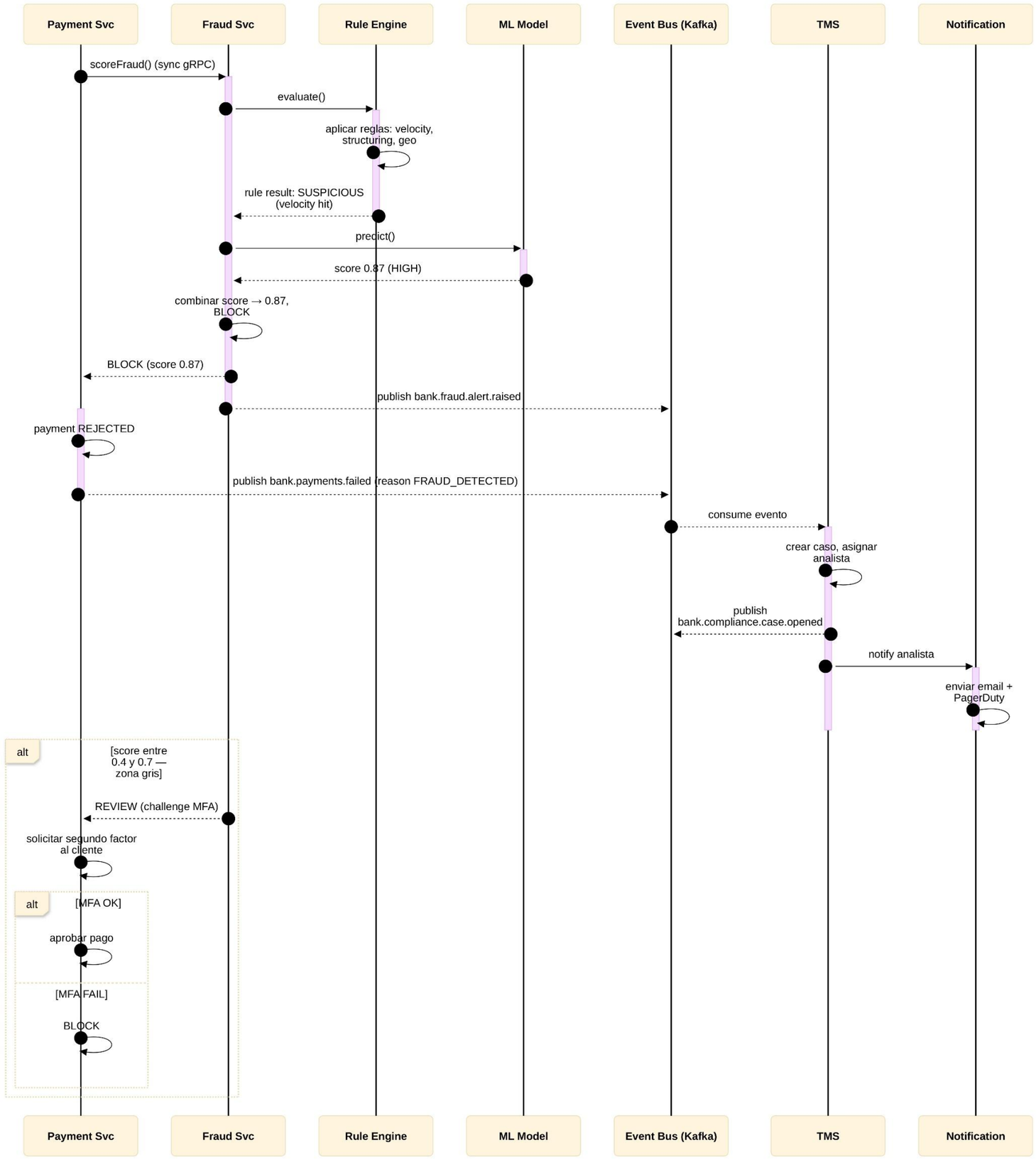
Onboarding de Cliente — KYC y CDD



Detección de Fraude en Tiempo Real

Score ML	Decision	Accion	Latencia max
0.00 – 0.39	CLEARED	Pago aprobado, flujo normal	<50ms
0.40 – 0.69	REVIEW	Challenge MFA al cliente. Si falla → BLOCK.	<50ms + MFA del usuario
0.70 – 1.00	BLOCK	Pago rechazado. Alerta TMS. Posible bloqueo de cuenta.	<50ms

Detección de Fraude en Tiempo Real



4. Patrones de integracion

Anti-Corruption Layer (ACL)			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
El Core Bancario Tradicional expone estructuras propietarias (IBM MQ, ISO 8583, SOAP) incompatibles con el modelo BIAN.	Cada microservicio dependería directamente del modelo del mainframe.	***LegacyCoreAdapter y DigitalCoreAdapter*** en Account Service (C4 L3, Sección 3). Ambos implementan la interfaz ***CoreBankingPort***	DIP (inversión de dependencias): los servicios dependen de la abstracción ***CoreBankingPort***, no de la implementación concreta. OCP: añadir un tercer core no modifica los servicios existentes.
Strangler Fig			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
El banco necesita modernizar su core sin interrumpir servicio a 100% de clientes existentes	Una migración big-bang exigiría migrar todos los clientes simultáneamente. Un fallo en la migración afectaría a la totalidad de la cartera del banco, con riesgo de pérdida de datos, multas regulatorias y daño reputacional irreversible.	Azure APIM actúa como fachada permanente (Sección 11). El feature flag ***USE_DIGITAL_CORE*** en Azure App Configuration determina el routing por segmento de cliente.	SRP: el APIM es responsable del routing, no de la lógica de negocio. Bajo acoplamiento: el frontend nunca sabe qué core está activo.
Saga (Orquestación)			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
Un pago involucra múltiples microservicios autónomos (Fraud, Risk, Account, SWIFT).	Sin Saga, un fallo en SWIFT dejaría el dinero debitado de la cuenta del cliente sin llegar al beneficiario.	***PaymentSagaOrchestrator*** en Payment Service (C4 L3 Payment Service, Sección 3.3). Define pasos: FraudCheck → RiskCheck → Debit → SWIFT. Cada paso tiene transacción	SRP: el APIM es responsable del routing, no de la lógica de negocio. Bajo acoplamiento: el frontend nunca sabe qué core está activo.

		compensatoria: si SWIFT falla, ejecuta ***RevertDebit***.	
CQRS			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
El Account Service debe atender simultáneamente operaciones transaccionales (débitos/créditos con consistencia fuerte) y millones de consultas de saldo (alta concurrencia, baja latencia).	Sin CQRS, las consultas de saldo de millones de clientes contenderían con las escrituras transaccionales sobre la misma base de datos.	Account Service y Payment Service (Sección 4B, C4 L3 Sección 3.4). ***AccountCommandHandler*** escribe al Event Store (Azure SQL). ***AccountQueryHandler*** lee de Cosmos DB (read model proyectado por ***AccountBalanceProjector***)	ISP: dos interfaces separadas ***ICommandPort*** e ***IQueryPort***. SRP: cada handler tiene una única responsabilidad.
Event Sourcing			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
Reguladores bancarios (BCBS, SBS, auditores) exigen historial inmutable y completo de cada transacción	Sin Event Sourcing, solo existe el estado actual en la base de datos. Ante una auditoría regulatoria o un litigio, sería imposible reconstruir la secuencia exacta de eventos que llevó a un saldo particular.	Event Store en Payment Service y Account Service (Sección 4B.2). Tabla ***payment_events*** append-only en Azure SQL — nunca UPDATE ni DELETE. El estado se reconstruye aplicando eventos en orden desde el stream.	SRP: el Event Store almacena hechos, los proyectores construyen vistas.

Outbox Pattern

Problema concreto que resuelve	Sin este patrón, ocurriría...	Componente exacto en el diseño	Principio reforzado
El Payment Service debe garantizar que cuando persiste un pago autorizado en su base de datos, el evento correspondiente en Kafka también se publique	Sin Outbox: si el servicio falla entre la escritura a la DB y la publicación en Kafka, el pago queda guardado pero ningún consumidor (Account Service para actualizar saldo, Notification Service para avisar al cliente) se entera.	***OutboxRepository*** en Payment Service y Account Service (C4 L3 Secciones 3.3 y 3.4). El evento y la entrada outbox se persisten en la misma transacción ACID.	SRP: el servicio de negocio no gestiona la publicación de eventos — esa responsabilidad está segregada en el publisher.
Circuit Breaker			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
El Fraud Detection Service puede degradarse bajo carga de ML o durante un pico de transacciones. Sin protección, una respuesta lenta de Fraud Detection bloquearía todos los threads del Payment Service	Sin Circuit Breaker: si Fraud Detection tarda 30s en responder, todos los threads del Payment Service quedan bloqueados esperando. En segundos, el Payment Service queda sin threads disponibles y comienza a rechazar todas las peticiones	Resilience4j en ***LegacyCoreAdapter***, llamadas a Fraud Detection y Risk Assessment (Secciones 4.X.1 y 9B.4). Umbral: 5 fallos en 10s → estado OPEN. Fallback conservador: DENY el pago. HALF-OPEN tras 30s para recovery.	SRP: la política de resiliencia está en el adapter/cliente, no en el servicio llamado. OCP: añadir Circuit Breaker no modifica el Fraud Service ni el Payment Service core.

Dead Letter Queue (DLQ)			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**

Los consumidores de Kafka pueden fallar al procesar un evento por razones transitorias (DB temporalmente no disponible) o permanentes (evento malformado, bug).	Sin DLQ: si el Account Service no puede procesar un evento bank.payments.completed, tiene dos opciones malas: (1) ignorarlo y perder la actualización de saldo (2) bloquearse en ese mensaje y dejar de procesar todos los eventos posteriores.	Topic ***bank.{dominio}.dlq*** en Kafka/Azure Event Hubs (Sección 6.3). Política: 3 reintentos con backoff exponencial (inmediato → 1s → 5s). Fallo tras 3 → DLQ + alerta Azure Monitor + alerta inmediata a operaciones para análisis y replay manual.	SRP: el flujo principal de procesamiento no gestiona reintentos — esa lógica está segregada. El consumer tiene una única responsabilidad: procesar el mensaje. Si no puede, delega al sistema de recuperación.

API Gateway (APIM Externo + Interno)			
Problema concreto que resuelve	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**
Sin un punto de entrada centralizado, cada microservicio debería implementar autenticación, rate limiting, WAF, logging, correlación y versionado individualmente	Sin API Gateway: cada microservicio expuesto directamente a internet implementa su propia autenticación. Un microservicio mal configurado expone directamente una vulnerabilidad.	Azure APIM Externo (clientes, Open Finance) y APIM Interno (inter-servicios). El APIM aplica: validación JWT, rate limiting por ***client_id***, WAF (OWASP Top 10), propagación del ***X-Correlation-ID***, routing por versión de API. Zero lógica de negocio bancario — solo cross-cutting concerns.	SRP: el flujo principal de procesamiento no gestiona reintentos — esa lógica está segregada. El consumer tiene una única responsabilidad: procesar el mensaje. Si no puede, delega al sistema de recuperación.

API Gateway (APIM Externo + Interno)			
Strategy Pattern (CoreRouter)	**Sin este patrón, ocurriría...**	**Componente exacto en el diseño**	**Principio reforzado**

Sin un punto de entrada centralizado, cada microservicio debería implementar autenticación, rate limiting, WAF, logging, correlación y versionado individualmente	Sin Strategy: cada servicio que necesite operar sobre una cuenta debería contener un bloque if (coreType == LEGACY) {...} else {...}. Al agregar un tercer core (p.ej. un proveedor BaaS)	***CoreRouter*** en Account Service (C4 L3 Sección 3.4). Resuelve en runtime qué implementación de ***CoreBankingPort*** usar según ***account.coreType.*** Al decommissionar el Core Legacy, solo se elimina el ***LegacyCoreAdapter*** — ningún otro componente cambia.	OCP: añadir un nuevo core = añadir un nuevo adapter, no modificar el router ni los servicios existentes. LSP: ***LegacyCoreAdapter*** y ***DigitalCoreAdapter*** son intercambiables bajo la misma interfaz. ISP: ***CoreBankingPort*** define solo los métodos necesarios para el dominio de cuenta.

Principio SOLID en los patrones

Los ACL implementan *LSP* (el cliente usa la misma interfaz CoreBankingPort sin importar si habla con legacy o digital). El Circuit Breaker y el Outbox son extensiones *OCP*: añaden comportamiento sin modificar los servicios core. El DLQ aplica *SRP*: el flujo principal no gestiona reintentos, esa responsabilidad está segregada. La tabla anterior explicita estas relaciones por cada patrón — los patrones no son decoración arquitectónica, son respuestas a problemas de negocio concretos.

Decisión de Diseño: Llamadas Síncronas vs Asíncronas en el Critical Path de Pago

Contexto de la decisión

el ejercicio advierte contra *"cadenas síncronas excesivas"*. El flujo de autorización de pago incluye dos llamadas síncronas consecutivas: Payment Service → Fraud Detection Service y Payment Service → Risk Assessment Service. Esta sección justifica explícitamente por qué son síncronas y no asíncronas, y qué controles evitan que se conviertan en un problema.

Criterio de decisión	Opción A: Síncrono (elegida)	Opción B: Asíncrono (descartada)
Semántica de negocio	El pago *no puede autorizarse* sin una decisión de fraude y riesgo. El resultado es una precondición bloqueante, no una notificación posterior.	Asíncrono implicaría autorizar el pago de forma provisional y esperar el resultado — lo que crea una ventana donde un pago fraudulento ya fue ejecutado antes de detectarlo.
Latencia real	Fraud Detection: p99 <50ms. Risk Assessment: p99 <100ms. Total de ambas en serie: <150ms — dentro del SLO global de <3s. Las llamadas son rápidas por diseño (modelos en memoria, Redis para features).	Un patrón async con callback (correlation ID + espera en Kafka) añadiría mínimo 200–500ms de overhead de mensajería, aumentando la latencia total sin beneficio.
Complejidad de implementación	gRPC unario estándar. Timeout explícito. Resultado inmediato. Simple.	Requiere: publicar request en Kafka, consumir respuesta correlacionada, manejar timeouts de respuesta parcial, gestionar estado intermedio — complejidad injustificada para una operación de <150ms.
Consistencia	La decisión de fraude y riesgo forman parte de la misma unidad lógica de autorización. Sync garantiza que son evaluadas con el mismo estado del sistema.	Async introduce riesgo de race condition: el estado puede cambiar entre la publicación del request y la recepción de la respuesta (otro pago concurrente de la misma cuenta).

¿Por qué no es una "cadena síncrona excesiva"?

Una cadena síncrona problemática tiene tres características:

1) muchos saltos

2) servicios lentos en el camino

3) acoplamiento innecesario

Este diseño evita las tres: solo hay dos llamadas síncronas en el critical path, ambas tienen SLO de latencia medida en decenas de milisegundos, y la dependencia es semánticamente necesaria (no es conveniencia de implementación). Las llamadas síncronas *restantes* en el sistema (notificaciones, reporting, actualización de saldo post-pago) son deliberadamente asíncronas vía Kafka.

Controles que Previenen Cascada de Fallos en las Llamadas Síncronas

Control	Configuración	Comportamiento ante fallo del servicio llamado
Timeout estricto	Fraud Detection: timeout 200ms. Risk Assessment: timeout 300ms.	Si el servicio no responde en el timeout, la llamada falla inmediatamente — no bloquea el thread indefinidamente.
Circuit Breaker (Resilience4j)	Umbral OPEN: 5 timeouts en 10s. Estado HALF-OPEN tras 30s.	Si Fraud Detection entra en OPEN: el pago es rechazado con código FRAUD_SERVICE_UNAVAILABLE. Política conservadora — ante duda, no autorizar.
Bulkhead (thread pool isolation)	Pool dedicado por servicio: 10 threads para Fraud, 10 para Risk. Separado del pool de requests de clientes.	Si Fraud Detection satura su pool (servicio lento), no afecta a otros endpoints del Payment Service ni a peticiones de consulta de saldo.
Fallback conservador	Si el circuit breaker está OPEN o el timeout se alcanza: decisión por defecto = DENY.	El banco prefiere rechazar un pago legítimo (falso negativo operacional) antes que aprobar uno fraudulento ante una falla técnica. Este es un requisito de negocio explícito, no una decisión técnica.
Llamadas en paralelo	Fraud Detection y Risk Assessment se llaman en **paralelo** (CompletableFuture / reactive), no en serie.	El tiempo total no es 50ms + 100ms = 150ms, sino **max(50ms, 100ms) = 100ms**. Las dos llamadas ocurren simultáneamente, reduciendo la latencia percibida a la del servicio más lento.

```
// Implementación: llamadas paralelas con timeout independiente
// Payment Service — PaymentAuthorizationOrchestrator.java

CompletableFuture<FraudCheckResult> fraudFuture =
    CompletableFuture.supplyAsync(() ->
        fraudClient.checkFraud(paymentRequest),    // gRPC, timeout 200ms
        fraudThreadPool                             // bulkhead: pool dedicado
    );

CompletableFuture<RiskCheckResult> riskFuture =
    CompletableFuture.supplyAsync(() ->
        riskClient.assessRisk(paymentRequest),     // gRPC, timeout 300ms
        riskThreadPool                             // bulkhead: pool dedicado
    );

// Espera ambos resultados en paralelo — latencia = max(fraud, risk)
CompletableFuture.allOf(fraudFuture, riskFuture)
    .orTimeout(350, TimeUnit.MILLISECONDS)         // timeout global conservador
    .join();

FraudCheckResult fraud = fraudFuture.get();
RiskCheckResult  risk  = riskFuture.get();

if (fraud.isBlocked() || risk.isDenied()) {
    return PaymentDecision.REJECTED;               // fallo conservador
}
```

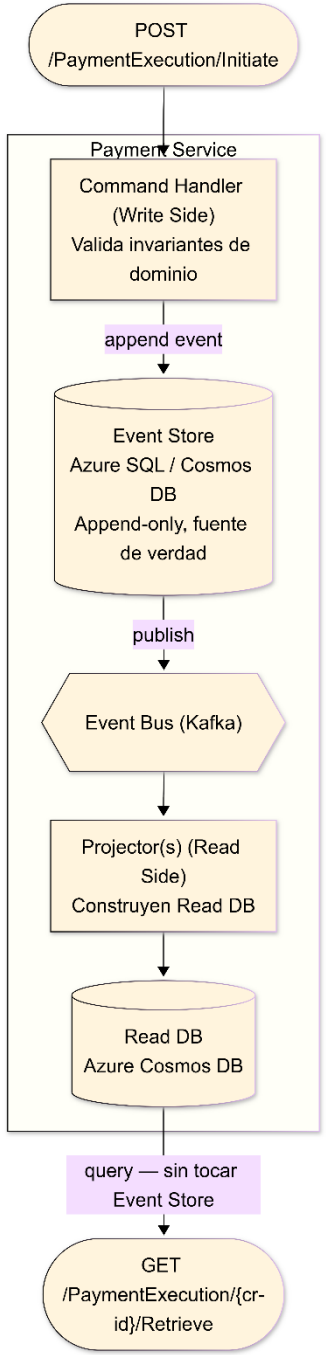
CQRS y Event Sourcing — Implementación Detallada

Decisión arquitectural: CQRS (Command Query Responsibility Segregation) y Event Sourcing se aplican exclusivamente en los dominios donde el historial de estado es un requisito de negocio o regulatorio.

Payment Service (auditoría inmutable de transacciones) y **Account Service** (historial de saldo y movimientos).

No se aplica universalmente — servicios como Notification o Reporting usan CRUD convencional.

Lado	Responsabilidad	Base de datos	Modelo de consistencia
Write Side (Command)	Validar reglas de negocio, persistir eventos en Event Store, publicar en Kafka	Azure SQL (Event Store — append-only, nunca UPDATE/DELETE)	Fuerte (transaccional, ACID)
Read Side (Query)	Proyectar eventos en vistas optimizadas para consulta. Responder GET requests.	Azure Cosmos DB (desnormalizado, optimizado por query)	Eventual (lag típico <100ms desde evento)



Event Sourcing — Estructura del Event Store

```
// Tabla: payment_events (Azure SQL — append-only)
// Nunca se hace UPDATE ni DELETE sobre esta tabla.

CREATE TABLE payment_events (
  sequence_number  BIGINT          IDENTITY PRIMARY KEY,    -- global ordering
  stream_id        NVARCHAR(64)    NOT NULL,                -- PAY-UUID-V4 (aggregate ID)
  stream_version   INT             NOT NULL,                -- versión dentro del stream
  event_type       NVARCHAR(128)   NOT NULL,                -- PaymentInitiated, FraudCleared...
  event_data       NVARCHAR(MAX)   NOT NULL,                -- JSON payload del evento
  metadata         NVARCHAR(MAX)   NOT NULL,                -- correlationId, userId, timestamp
  created_at       DATETIMEOFFSET  NOT NULL DEFAULT SYSUTCDATETIME(),
  causation_id     NVARCHAR(64)    NOT NULL,                -- ID del comando que causó el evento
  UNIQUE (stream_id, stream_version)                        -- optimistic concurrency
);

-- Índice para carga de stream por aggregate
CREATE INDEX idx_payment_events_stream ON payment_events (stream_id, stream_version);
```

Ciclo de Vida de un Pago en Event Sourcing

stream_version	event_type	Estado derivado del aggregate
1	PaymentInitiated	INITIATED — pago creado, pendiente validación
2	FraudCheckCleared	FRAUD_CLEARED — score ML < 0.4
3	RiskCheckApproved	RISK_APPROVED — límites y reglas OK
4	DebitExecuted	DEBITED — core bancario confirmó débito
5	SwiftMessageSent	IN_TRANSIT — pacs.008 enviado a corresponsal
6	PaymentCompleted	COMPLETED — pacs.002 ACCP recibido

El estado actual del pago **no se almacena**, se deriva siempre aplicando los eventos en orden sobre el aggregate vacío. Esto garantiza que el historial es la única fuente de verdad y es 100% auditable.

Snapshot Strategy — Rendimiento a Escala

Sin snapshots, reconstruir el estado de un aggregate requiere cargar todos sus eventos desde el origen. Para un pago con 6 eventos esto es trivial. Para una **cuenta corriente** con 10 años de movimientos (miles de eventos), es inaceptable. La estrategia de snapshots resuelve esto.

```
// Snapshot: fotografía del estado del aggregate en un punto dado
// Tabla: account_snapshots (Azure SQL)

CREATE TABLE account_snapshots (
  account_id       NVARCHAR(64)    NOT NULL,
  stream_version   INT             NOT NULL,   -- versión del último evento incluido
  state_data       NVARCHAR(MAX)   NOT NULL,   -- JSON del estado completo del aggregate
  created_at       DATETIMEOFFSET  NOT NULL DEFAULT SYSUTCDATETIME(),
  PRIMARY KEY (account_id, stream_version)
);

// Estrategia de toma de snapshots:
// 1. Cada 50 eventos sobre el mismo stream (configurable por aggregate type)
// 2. Al cierre del día (COB) para todas las cuentas activas
// 3. Antes de una migración de cuenta (legacy → digital)

// Reconstrucción del aggregate con snapshot:
// 1. Cargar el snapshot más reciente (state_data, stream_version = N)
// 2. Cargar solo los eventos con stream_version > N
// 3. Aplicar esos eventos sobre el estado del snapshot
// → Máximo 50 eventos a procesar, independientemente de la antigüedad de la cuenta
```

Proyectores — Read Models por Caso de Uso

Proyector	Consume eventos	Read Model (Cosmos DB)	Query que sirve
PaymentStatusProjector	PaymentInitiated, PaymentCompleted, PaymentFailed	Colección payment_status: {paymentId, status, amount, timestamps}	GET /PaymentExecution/{cr-id}/Retrieve
AccountBalanceProjector	DebitExecuted, CreditExecuted, InterestAccrued	Colección account_balance: {accountId, availableBalance, bookBalance, lastUpdated}	GET /CurrentAccount/{cr-id}/Balance/{bq-id}/Retrieve
TransactionHistoryProjector	Todos los eventos de movimiento de cuenta	Colección transaction_history: lista paginada de movimientos con metadatos	GET /CurrentAccount/{cr-id}/Transactions/{bq-id}/Retrieve
FraudAuditProjector	FraudCheckCleared, FraudCheckBlocked, FraudAlertRaised	Colección fraud_audit: historial de decisiones para reguladores y analistas	Query interna Compliance / Reporting Service

Stream Versioning — Evolución de Eventos sin Breaking Changes

```
// Problema: ¿cómo añadir un campo nuevo a PaymentInitiated sin romper
// los proyectores que consumen eventos históricos del Event Store?

// Estrategia: Upcaster (transformación de versión en el pipeline de lectura)

// v1 — evento original (en Event Store histórico):
{ "type": "PaymentInitiated", "version": 1,
  "amount": 500.00, "currency": "USD", "debtorAccountId": "ACC-123" }

// v2 — nueva versión (añade purposeCode para ISO 20022 compliance):
{ "type": "PaymentInitiated", "version": 2,
  "amount": 500.00, "currency": "USD", "debtorAccountId": "ACC-123",
  "purposeCode": "TRAD" }

// Upcaster: transforma v1 → v2 en memoria antes de aplicar al aggregate
// Los eventos v1 históricos NO se modifican en el Event Store
class PaymentInitiatedUpcaster {
    PaymentInitiatedV2 upcast(PaymentInitiatedV1 event) {
        return new PaymentInitiatedV2(event, purposeCode: "UNKNOWN");
    }
}

// Regla: el Event Store es inmutable. Los upcasters son la única
// forma de adaptar eventos históricos a esquemas nuevos.
```

Reconstrucción de Proyector Caído

Si un proyector falla o necesita ser reconstruido (cambio de esquema del read model), el proceso es determinista y sin pérdida de datos:

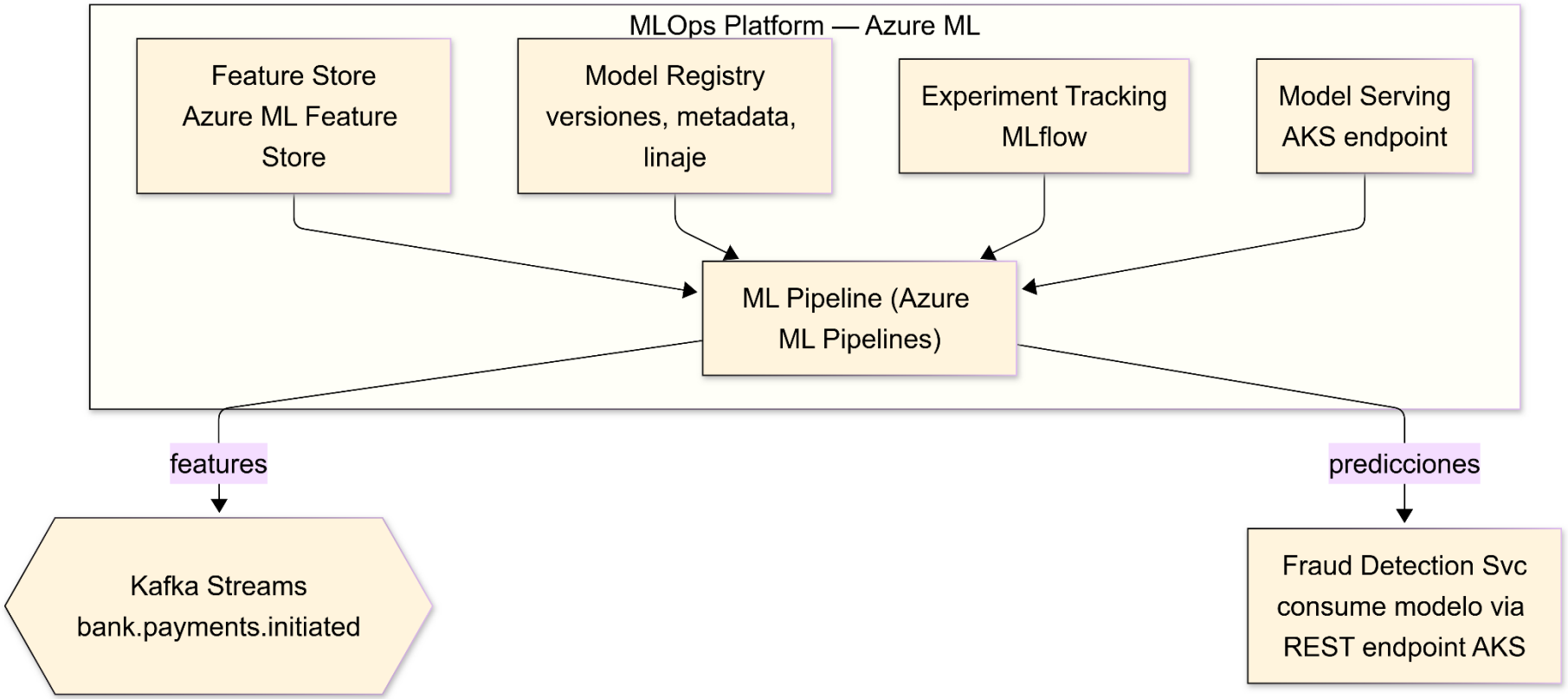
- Borrar la colección Cosmos DB del proyector afectado (read model, no Event Store).
- Reiniciar el proyector apuntando al offset 0 del topic Kafka correspondiente (o al inicio del Event Store).
- El proyector reprocesa todos los eventos en orden. Los upcasters adaptan versiones históricas.
- Mientras reconstruye, las queries sirven datos del read model anterior (si existe) o retornan ***202 Rebuilding*** con ***Retry-After*** header.
- Al completar la reconstrucción, el proyector publica un evento interno ***ProjectorReady*** y el servicio empieza a servir el nuevo read model.

MLOps — Ciclo de Vida del Modelo de Detección de Fraude

Un modelo ML de fraude sin gobierno operacional se degrada silenciosamente: el patrón de fraude evoluciona, el modelo no se actualiza, y la tasa de falsos negativos sube hasta que el banco detecta el problema por pérdidas reales.

Esta sección define la arquitectura MLOps que garantiza que el modelo en producción siempre esté actualizado, explicable y auditable por reguladores.

Arquitectura MLOps — Componentes



Feature Store — Features de Detección de Fraude

Feature	Descripción	Ventana temporal	Tipo
tx_count_24h	Número de transacciones del cliente en las últimas 24h	24 horas	Online (Redis)
tx_amount_sum_1h	Suma de montos de transacciones en la última hora	1 hora	Online (Redis)
avg_tx_amount_30d	Monto promedio histórico de transacciones del cliente (30 días)	30 días	Offline (Azure ML Feature Store)
is_new_beneficiary	El beneficiario es nuevo para este cliente (nunca pagado antes)	Histórico completo	Online (Redis lookup)
geo_distance_km	Distancia geográfica entre la ubicación del dispositivo y la última transacción	Última TX	Online (real-time)
device_fingerprint_match	¿El device fingerprint coincide con los dispositivos conocidos del cliente?	Histórico 90 días	Online (Redis)
hour_of_day	Hora de la transacción (0–23) — detecta actividad fuera del patrón horario	Real-time	Online (real-time)
beneficiary_risk_score	Score de riesgo acumulado del beneficiario (basado en transacciones anteriores)	Histórico completo	Offline (batch nocturno)
sanctions_hit	El beneficiario aparece en alguna lista de sanciones	Real-time	Online (Sanctions Screening API)

Model Registry — Versionado y Linaje

```
// Azure ML Model Registry — ejemplo de metadata de un modelo registrado

Model: fraud-detection-xgboost
Version: 3.2.1
Stage: Production (Champion)
Registered: 2026-05-10T08:00:00Z
Trained by: ML Pipeline #441 (Azure ML)

Training dataset: transactions_2024-01_2026-04 (18 meses de datos)
Features used: [tx_count_24h, tx_amount_sum_1h, avg_tx_amount_30d,
                is_new_beneficiary, geo_distance_km, device_fingerprint_match,
                hour_of_day, beneficiary_risk_score, sanctions_hit]

Métricas de evaluación (test set, holdout 3 meses):
  AUC-ROC:          0.9847
  Precision (0.5):  0.923
  Recall (0.5):     0.891
  F1-Score:         0.907
  False Positive Rate: 0.034  (3.4% de transacciones legítimas bloqueadas)
  False Negative Rate: 0.109  (10.9% de fraudes no detectados)

Thresholds operacionales (ver 3B.3):
  CLEARED:  score < 0.40
  REVIEW:   score 0.40–0.69
  BLOCK:    score ≥ 0.70

Linaje de datos:  Kafka → Azure Data Lake → Feature Store → Training Job → Model
Aprobado por:     ML Lead + Risk Officer + Compliance Officer
```

Champion / Challenger Framework

En producción siempre conviven dos modelos: el **Champion** (modelo vigente, maneja el 95% del tráfico) y el **Challenger** (nuevo candidato, maneja el 5% del tráfico en A/B). El Challenger solo reemplaza al Champion si supera sus métricas durante un período de observación mínimo de 30 días.

Parámetro	Champion (v3.2.1)	Challenger (v3.3.0)
Tráfico asignado	95%	5% (shadow mode + live)
Algoritmo	XGBoost	LightGBM (candidato más eficiente)
AUC-ROC	0.9847	0.9891 (en evaluación)
Latencia p99	12ms	8ms (más rápido)
Criterio de promoción	Challenger supera Champion en AUC-ROC AND reduce FPR AND mantiene latencia <50ms durante 30 días en producción	
Rollback	Si Challenger degrada: tráfico 100% a Champion en <5 min (feature flag en Azure App Configuration)	

Explicabilidad de Decisiones (Requisito Regulatorio)

Los reguladores financieros (y la GDPR Art. 22) exigen que las decisiones automatizadas que afecten a un cliente sean explicables. Si un pago es bloqueado por el modelo ML, el banco debe poder explicar a qué se debe — no solo el score, sino las features que más contribuyeron a la decisión.

```
// SHAP (SHapley Additive exPlanations) — explicabilidad por transacción

// Ejemplo: pago bloqueado (score: 0.87)
{
  "paymentId":    "PAY-UUID-1234",
  "fraudScore":   0.87,
  "decision":     "BLOCK",
  "explanation": {
    "method": "SHAP",
    "baseScore": 0.15,          // score base (tasa base de fraude del portafolio)
    "contributions": [
      { "feature": "is_new_beneficiary",       "value": 1,       "shap": +0.31 },
      { "feature": "tx_amount_sum_1h",         "value": 4800.00, "shap": +0.22 },
      { "feature": "geo_distance_km",          "value": 1240,    "shap": +0.18 },
      { "feature": "device_fingerprint_match", "value": 0,       "shap": +0.09 },
      { "feature": "avg_tx_amount_30d",        "value": 85.00,   "shap": -0.08 }
    ],
    "humanReadable": "Pago bloqueado: beneficiario desconocido (+0.31), monto inusualmente alto en la última hora (+0.22) y transacción iniciada desde ubicación lejana a la habitual (+0.18)."
  }
}
```

5. Estrategia de APIs Internas y Externas

APIs Internas (Inter-Servicio)

Dimensión	Decisión	Justificación
Protocolo	REST (HTTPS) + gRPC para comunicación de baja latencia	REST para operaciones de negocio; gRPC (Protocol Buffers) para llamadas en tiempo real como fraud check (<50ms)
Contrato	OpenAPI 3.0 (REST) / Protobuf (gRPC)	Contract-first: la spec se define antes del código y se versiona en Git
Seguridad	mTLS + JWT (15 min TTL) validado en APIM Interno	Zero Trust: cada microservicio prueba su identidad con certificado cliente
Versionado	Path versioning: /v1/, /v2/	Permite coexistencia de versiones durante migración sin breaking changes
Testing contratos	Pact (Consumer-Driven Contract Testing)	El consumidor define las expectativas; el proveedor las verifica en CI/CD

APIs Externas (Open Finance / Canales)

Dimensión	Decisión	Justificación
Estándar	FAPI 1.0 Advanced (Financial-grade API)	Requerido por regulaciones Open Finance / PSD2. Construido sobre OAuth 2.0 + PKCE + PAR
Seguridad	OAuth 2.0 + PKCE + scopes granulares	Scopes: accounts:read, payments:write, statements:read. Cada proveedor solo accede a lo autorizado explícitamente
Rate limiting	Por client_id + por IP + por plan de suscripción	Previene abuso. Plan Free: 100 req/min; Plan Premium: 1000 req/min
Versionado	Semver en path + deprecation policy 6 meses	Los proveedores Open Finance tienen SLAs propios; necesitan aviso con antelación
Developer Portal	Azure APIM Developer Portal con sandbox	Reduce fricción para onboarding de fintechs; sandbox con datos sintéticos

Ejemplos de APIs — Formato BIAN v14 (Release 14.0.0)

Convención BIAN oficial: /{ServiceDomain} /{cr-reference-id} /{BehaviorQualifier} /{bq-reference-id} /{ActionTerm}
Los 8 Action Terms estándar: Initiate · Update · Control · Execute · Request · Retrieve · Exchange · Notify

Consulta de Saldo — CurrentAccount / Retrieve (API Interna)

```
// BIAN Service Domain: Current Account
// Control Record: CurrentAccountFulfillment
// Behavior Qualifier: Balance
// Action Term: Retrieve

GET /CurrentAccount/{cr-reference-id}/Balance/{bq-reference-id}/Retrieve
Authorization: Bearer {jwt_servicio}   // mTLS + JWT interno — validado en APIM Interno
X-Correlation-ID: corr-550e8400-e29b-41d4-a716-446655440000

HTTP/1.1 200 OK
{
  "currentAccountFulfillmentArrangement": {
    "accountNumber":      "ACC-00234567",
    "iban":               "ES7621000813610123456789",
    "accountCurrency":    "USD",
    "coreSource":         "DIGITAL"
  },
  "balance": {
    "balanceType":        "Available",
    "amount":             14920.50,
    "currency":           "USD",
    "valuationDate":      "2026-06-23T10:30:00Z"
  }
}
```

Iniciación de Pago — PaymentExecution / Initiate (API Interna)

```
// BIAN Service Domain: Payment Execution
// Control Record: PaymentExecutionProcedure
// Action Term: Initiate  (crea y activa el proceso de pago — patrón Fulfill)

POST /PaymentExecution/Initiate
Authorization: Bearer {jwt_servicio}
X-Idempotency-Key: {uuid-v4}           // Idempotency Handler lo valida contra Redis
X-Correlation-ID: corr-550e8400-...

{
  "paymentExecutionProcedure": {
    "paymentTransactionInitiatorReference": "E2E-2026-0623-001",
    "paymentType":                          "CreditTransfer",
    "paymentAmount":        { "amount": "500.00", "currency": "USD" },
    "paymentMechanism":     "SWIFT",
    "debtorAccount":        { "iban": "ES9121000418450200051332" },
    "creditorAccount":      { "iban": "ES7621000813610123456789" },
    "paymentPurpose":       "Factura #2026-0089",
    "requestedExecutionDate": "2026-06-23"
  }
}

HTTP/1.1 201 Created
{
  "paymentExecutionProcedure": {
    "paymentExecutionProcedureInstanceReference": "PAY-UUID-V4",
    "paymentExecutionProcedureInstanceStatus":    "PENDING_FRAUD_CHECK"
  }
}
```

Consulta de Estado de Pago — PaymentExecution / Retrieve (API Interna)

```
// Action Term: Retrieve  (consulta el Control Record existente)

GET /PaymentExecution/{cr-reference-id}/Retrieve
Authorization: Bearer {jwt_servicio}

HTTP/1.1 200 OK
{
  "paymentExecutionProcedure": {
    "paymentExecutionProcedureInstanceReference": "PAY-UUID-V4",
    "paymentExecutionProcedureInstanceStatus":    "AUTHORIZED",
    "paymentMechanism":   "SWIFT",
    "paymentAmount":      { "amount": "500.00", "currency": "USD" },
    "valueDate":          "2026-06-23T10:31:00Z",
    "fraudCheckResult":   "CLEARED",
    "riskScore":          0.12
  }
}
```

Autorización Open Finance — PaymentExecution / Execute (API Externa, FAPI 1.0)

```
// Expuesta vía APIM Externo bajo perfil FAPI 1.0 Advanced
// Action Term: Execute  (ejecuta la instrucción de pago ya iniciada)

POST /PaymentExecution/{cr-reference-id}/PaymentInstruction/{bq-reference-id}/Execute
Authorization: Bearer {fapi_token}     // OAuth 2.0 + PKCE + mTLS client certificate
X-Idempotency-Key: {uuid-v4}
X-Correlation-ID: corr-...

{
  "paymentInstruction": {
    "paymentInstructionType":         "Execute",
    "paymentInstructionAuthorizationCode": "AUTH-2026-0089",
    "confirmationRequested":          true
  }
}

HTTP/1.1 200 OK
{
  "paymentInstruction": {
    "paymentInstructionReference":    "PAY-INSTR-001",
    "paymentInstructionStatus":       "EXECUTED",
    "executionDateTime":              "2026-06-23T10:31:45Z",
    "endToEndReference":              "E2E-2026-0623-001"
  }
}
```

Alerta de Fraude — FraudDiagnosis / Notify (AsyncAPI — Evento)

```
// BIAN Service Domain: Fraud Diagnosis
// Action Term: Notify  (broadcast de cambio de estado — patrón asíncrono AsyncAPI 3.x)
// Canal: Kafka topic  bank.fraud.alerts

{
  "eventId":        "evt-fraud-uuid-v4",
  "eventType":      "FraudDiagnosis/Notify",
  "version":        "14.0",
  "correlationId":  "corr-550e8400-...",
  "source":         "fraud-detection-service",
  "partitionKey":   "ACC-00234567",
  "fraudDiagnosisAssessment": {
    "fraudDiagnosisAssessmentInstanceReference": "FRAUD-DIAG-001",
    "fraudDiagnosisEvaluation":  "HIGH_RISK",
    "fraudModel":                "ML-ENSEMBLE-v3",
    "fraudScore":                0.94,
    "triggeredRule":             "UNUSUAL_GEOGRAPHY + AMOUNT_SPIKE",
    "recommendedAction":         "BLOCK_AND_REVIEW",
    "assessmentDateTime":        "2026-06-23T10:30:43Z"
  }
}
```

6. Arquitectura de Eventos (Apache Kafka)

Estándar
Los eventos siguen **AsyncAPI 3.x** (estándar BIAN v14.0.0 para comunicación asíncrona). Los canales síncronos usan OpenAPI 3.x; los eventos y notificaciones usan AsyncAPI 3.x sobre Apache Kafka / Azure Event Hubs.

Definición de Canal AsyncAPI 3.x — Payment Execution / Notify

Ejemplo de definición AsyncAPI 3.x para el canal de pagos autorizados:

```
// asyncapi: 3.0.0
// BIAN Service Domain: Payment Execution — Action Term: Notify
// Canal Kafka: bank.payments.authorized

asyncapi: '3.0.0'
info:
  title: Payment Execution Events
  version: '14.0.0'
  description: Eventos asincrónicos del Service Domain Payment Execution (BIAN v14)

channels:
  bank.payments.authorized:
    address: bank.payments.authorized
    messages:
      PaymentAuthorizedNotify:
        $ref: '#/components/messages/PaymentAuthorizedNotify'

operations:
  onPaymentAuthorized:
    action: receive
    channel:
      $ref: '#/channels/bank.payments.authorized'
    messages:
      - $ref: '#/channels/bank.payments.authorized/messages/PaymentAuthorizedNotify'

components:
  messages:
    PaymentAuthorizedNotify:
      name: PaymentExecution/Notify
      title: Pago Autorizado — BIAN Payment Execution Notify
      contentType: application/json
      headers:
        type: object
        properties:
          X-Correlation-ID: { type: string, format: uuid }
          X-BIAN-Version:   { type: string, example: "14.0.0" }
          X-Partition-Key:  { type: string, description: "debtorAccountId para ordering" }
      payload:
        $ref: '#/components/schemas/PaymentExecutionNotifyPayload'
```

```yaml
schemas:
  PaymentExecutionNotifyPayload:
    type: object
    required: [eventId, eventType, version, timestamp, correlationId, partitionKey, paymentExecutionProcedure]
    properties:
      eventId:        { type: string, format: uuid, description: "Clave idempotencia — verificada en Redis TTL 24h" }
      eventType:      { type: string, example: "PaymentExecution/Notify" }
      version:        { type: string, example: "14.0.0" }
      timestamp:      { type: string, format: date-time }
      correlationId:  { type: string, format: uuid, description: "Propagado desde APIM — trazabilidad end-to-end" }
      source:         { type: string, example: "payment-service" }
      partitionKey:   { type: string, description: "debtorAccountId — garantiza ordering por cuenta en Kafka" }
      paymentExecutionProcedure:
        type: object
        properties:
          paymentExecutionProcedureInstanceReference: { type: string, example: "PAY-UUID-V4" }
          paymentExecutionProcedureInstanceStatus:    { type: string, example: "AUTHORIZED" }
          paymentAmount:    { type: number, example: 500.00 }
          paymentCurrency:  { type: string, example: "USD" }
          debtorAccountId:  { type: string, example: "ACC-00234567" }
          creditorAccountId:{ type: string, example: "ACC-00987654" }
          paymentMechanism: { type: string, example: "SWIFT" }
          fraudCheckResult: { type: string, example: "CLEARED" }
          fraudScore:       { type: number, example: 0.12 }
```

Ejemplo de Mensaje en Tiempo de Ejecución

```
// Mensaje publicado en Kafka topic:
bank.payments.authorized
// Formato: AsyncAPI 3.x — BIAN PaymentExecution/Notify

Headers:
  X-Correlation-ID: corr-550e8400-e29b-41d4-a716-
446655440000
  X-BIAN-Version:   14.0.0
  X-Partition-Key:  ACC-00234567

Payload:
{
  "eventId":       "evt-550e8400-e29b-41d4-a716-
446655440000",
  "eventType":     "PaymentExecution/Notify",
  "version":       "14.0.0",
  "timestamp":     "2026-06-23T10:30:45.123Z",
  "correlationId": "corr-550e8400-e29b-41d4-a716-
446655440000",
  "source":        "payment-service",
  "partitionKey":  "ACC-00234567",
  "paymentExecutionProcedure": {
    "paymentExecutionProcedureInstanceReference": "PAY-
UUID-V4",
    "paymentExecutionProcedureInstanceStatus":
"AUTHORIZED",
    "paymentAmount":     500.00,
    "paymentCurrency":   "USD",
    "debtorAccountId":   "ACC-00234567",
    "creditorAccountId": "ACC-00987654",
    "paymentMechanism":  "SWIFT",
    "fraudCheckResult":  "CLEARED",
    "fraudScore":        0.12
  }
}
```

Catálogo de Eventos Principales

Evento (topic)	Productor	Consumidores	Partición por
bank.customers.created	Customer Service	Account Svc, Notification Svc, Reporting Svc	customerId
bank.accounts.opened	Account Service	Reporting Svc, Notification Svc	accountId
bank.payments.initiated	Payment Service	Fraud Detection, Risk Assessment	debtorAccountId
bank.payments.authorized	Payment Service	ACL Adapter (Legacy/Digital), Reporting, Notification	debtorAccountId
bank.payments.completed	Payment Service	Account Svc (actualizar saldo), Notification Svc	debtorAccountId
bank.payments.failed	Payment Service	Notification Svc → DLQ si falla	debtorAccountId
bank.fraud.alert.raised	Fraud Detection Svc	Payment Service (bloquear), Risk Svc, Reporting	accountId
bank.customers.migrated	Migration Service	ACL Legacy (marcar migrado), Core Digital (crear)	customerId

Lineamientos de Calidad de Eventos

Lineamiento	Implementación
Idempotencia	Cada consumer persiste el eventId en Redis (TTL 24h). Si el evento ya fue procesado, se descarta sin efecto secundario.
Ordering	Partición por accountId: todos los eventos de una cuenta van a la misma partición → orden garantizado por cuenta.
Reintentos	Política exponential backoff: intento 1 (inmediato), 2 (1s), 3 (5s). Tras 3 fallos → DLQ.
Dead Letter Queue	Topic bank.{dominio}.dlq con alerta automática en Azure Monitor. Proceso de revisión manual + replay.
Versionado de Schema	Azure Schema Registry (Avro). Compatibilidad backwards por defecto. Cambios breaking → nueva versión del topic.
Payload de referencia	El evento lleva solo IDs + datos mínimos. Si el consumer necesita datos completos, hace GET al servicio propietario (evita duplicación y acoplamiento de datos).
Replication Factor	Factor 3, min.insync.replicas=2. Garantiza durabilidad ante fallo de un broker.

Integración SWIFT — ISO 20022 MX Messages

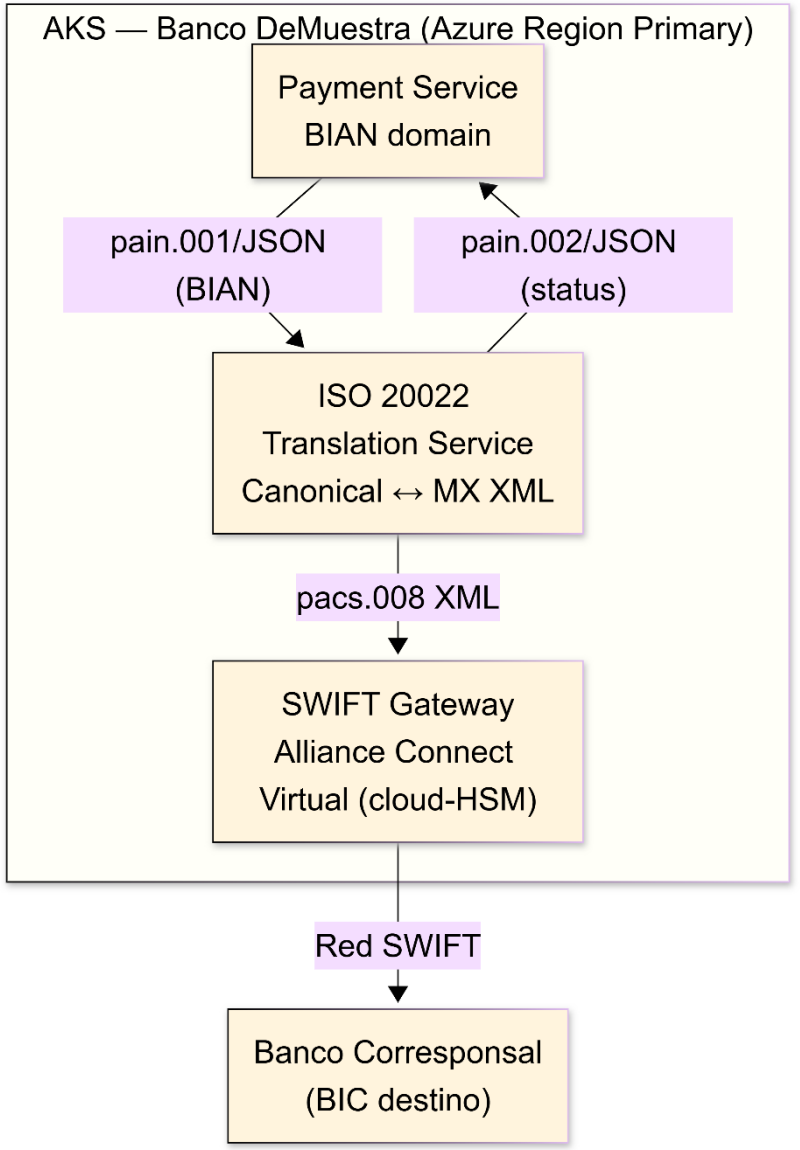
SWIFT completó en noviembre 2025 la migración obligatoria de mensajes MT (Message Text — formato propietario SWIFT, texto plano) a MX (Message XML — ISO 20022, XML estructurado). *MX no es una abreviación geográfica; es el estándar XML definido por ISO 20022, en contraposición al MT legacy.* Todo banco operando en la red SWIFT para pagos internacionales debe generar y consumir mensajes MX de forma nativa.

Familias de Mensajes ISO 20022 Utilizadas

Familia	Código	Mensaje	Flujo	MT Legacy equivalente
pain *Payment Initiation*	pain.001	CustomerCreditTransferInitiation	Cliente → Banco DeMuestra	MT101
	pain.002	CustomerPaymentStatusReport	Banco → Cliente (estado del pago)	MT900 / MT910
pacs *Payment Clearing & Settlement*	pacs.008	FIToFICustomerCreditTransfer	Banco DeMuestra → Banco Corresponsal (red SWIFT)	MT103
	pacs.002	FIToFIPaymentStatusReport	Banco Corresponsal → Banco DeMuestra	MT199 / NACK
	pacs.009	FinancialInstitutionCreditTransfer	Banco → Banco (cubrimiento interbancario)	MT202
camt *Cash Management*	camt.053	BankToCustomerStatement	Banco → Cliente (extracto de cuenta)	MT940
	camt.054	BankToCustomerDebitCreditNotification	Banco → Cliente (notificación de movimiento)	MT942

Arquitectura de Conectividad SWIFT en Azure

- **SWIFT Alliance Connect Virtual (SCV):** solución cloud-native de SWIFT para conectar desde Azure sin infraestructura on-premise (elimina el HSM físico propio). SCV integra con AKS vía Private Endpoint y gestiona la firma criptográfica de mensajes.
- **ISO 20022 Translation Service:** microservicio dedicado (ACL de protocolo) que transforma el modelo canónico interno BIAN JSON a mensajes MX (XML) y viceversa. El Payment Service nunca habla XML 20022 directamente.
- **Schema validation:** cada mensaje MX entrante se valida contra el XSD oficial ISO 20022 antes de procesar. Mensajes inválidos → DLQ con alerta inmediata a operaciones.
- **Firma de mensajes:** todos los mensajes salientes se firman con el certificado BIC de Banco DeMuestra, gestionado en Azure Key Vault HSM.



ISO

Flujo Completo — Transferencia Internacional (pacs.008)

#	Actor	Mensaje	Formato
1	Cliente inicia desde Web / Mobile	POST /PaymentExecution/Initiate	REST JSON (BIAN)
2	Payment Service orquesta Saga	FraudCheck → SanctionsScreening → RiskCheck → Authorize	Kafka eventos internos
3	Payment Service → Translation Svc	Solicitud de generación MX	gRPC (modelo canónico)
4	Translation Svc genera mensaje	pacs.008.001.10 (versión ISO 20022 MX vigente)	XML firmado (Key Vault)
5	SWIFT Gateway envía a red	pacs.008 → Banco corresponsal	XML / red SWIFT
6	Banco corresponsal responde	pacs.002 (ACCP / RJCT / PDNG)	XML
7	Translation Svc convierte respuesta	Status → modelo canónico JSON	gRPC
8	Payment Service publica evento	bank.payments.completed o bank.payments.failed	Kafka (AsyncAPI 3.x)
9	Notification Service notifica cliente	Push notification / email	Azure Notification Hubs

Campos Clave en MX vs MT Legacy

Campo ISO 20022 MX	**Campo ISO 20022 MX**	**Impacto regulatorio / operativo**
StructuredPostalAddress	Dirección desglosada por campos (calle, ciudad, país, CP) — no texto libre	Sanctions screening preciso — elimina falsos positivos por mala normalización de texto
LEI (Legal Entity Identifier — ISO 17442)	Identificador único de la entidad legal originante	Mandatorio para pagos corporativos en jurisdicciones G20. Trazabilidad de entidad legal.
PurposeCode	Código ISO del propósito del pago (SALA, DIVI, TRAD, SUPP…)	AML/CFT: detección de patrones por tipo de pago. Requerido por reguladores de muchas jurisdicciones.
InstrPrty (Instruction Priority)	NORM / HIGH — urgencia de procesamiento	SLA diferenciado: HIGH procesados en <30 min; NORM en T+1. Pricing diferente.
RmtInf/Ustrd	Información de remesa estructurada	Reconciliación automática en banco beneficiario — elimina matching manual.
ChrgBr (Charge Bearer)	DEBT / CRED / SHAR / SLEV — quién paga los fees bancarios	Claridad contractual en fees de corresponsalía. Evita disputas post-liquidación.

Manejo de Rechazos — R-Messages (pacs.002)

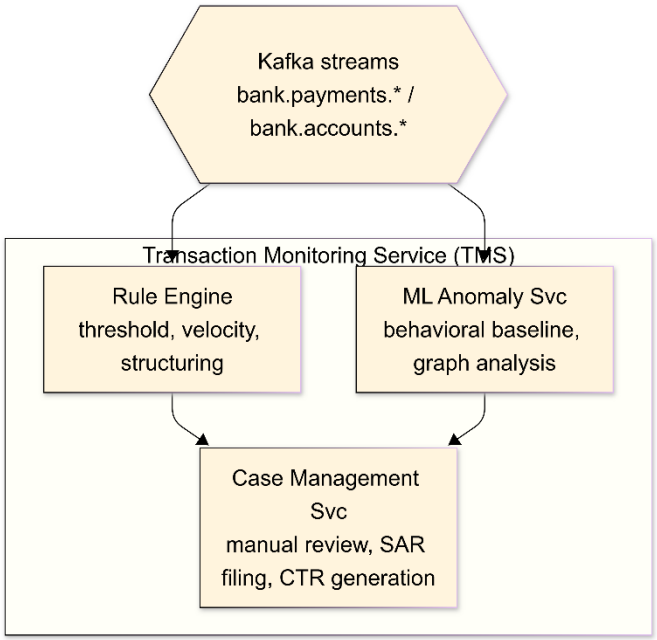
Reason Code	Significado	Acción en Payment Service
RJCT / AC01	IBAN del beneficiario incorrecto	Publicar ***bank.payments.failed*** (reason: AC01). Notificar cliente. No reintentar automáticamente.
RJCT / AM04	Fondos insuficientes en cuenta nostro del corresponsal	Alerta al equipo de nostro management. Intentar ruta de corresponsal alternativa si existe.
RJCT / FF01	Formato del mensaje inválido (XSD violation)	DLQ + alerta crítica P1 al equipo de integración. Indica defecto en Translation Service.
RJCT / NARR	Rechazo con narrativa libre (AML, sanctions hit)	Escalar a equipo Compliance. Registrar en sistema AML. Bloquear cuenta si procede. No reintentar.
PDNG	Pago pendiente en corresponsal — sin confirmación	Polling estado a las 4h. Si sin resolución a las 24h: enviar camt.056 (solicitud de cancelación).

Arquitectura AML/CFT — Antilavado de Dinero y Financiamiento del Terrorismo

Marco regulatorio

La arquitectura cumple con las 40 Recomendaciones del GAFI/FATF, con énfasis en Rec. 10 (Customer Due Diligence), Rec. 16 (Travel Rule — información del originante en transferencias), y Rec. 20 (Reporte de Operaciones Sospechosas). El enfoque es *Risk-Based Approach (RBA)*: los controles se calibran según el perfil de riesgo del cliente y la tipología de transacción.

Arquitectura del Transaction Monitoring System (TMS)



Tipologías de Detección — Rule Engine

Tipología AML	Regla de detección	Umbral / Parámetro	Acción
Structuring (Pitufeo / Smurfing)	Múltiples transacciones fraccionadas por debajo del umbral de reporte CTR, sumando un total por encima del umbral en ventana de tiempo	N transacciones < $9,999 sumando > $10,000 en 24h por mismo originante	Alerta alta prioridad. Bloquear cuenta pendiente de revisión.
Layering (Capas)	Transferencias en cadena entre múltiples cuentas sin propósito económico aparente, con variación mínima de monto	> 3 transferencias relacionadas en <1h con <5% variación de monto	Alert + análisis de grafo de relaciones entre cuentas (graph ML).
Round-Tripping	Fondos que salen y regresan a la cuenta originante a través de cuentas intermedias	Retorno de >80% del monto original en <72h	Alerta + congelamiento preventivo + escalada a Compliance.
Velocity Anomaly	Volumen de transacciones significativamente por encima del baseline histórico del cliente	> 3 desviaciones estándar del promedio mensual en 7 días	Alerta media. Revisión por analista. EDD si confirma.
Geographic Anomaly	Transacción desde país de alto riesgo FATF (lista gris / negra) o jurisdicción no cooperante	BIC o IP del beneficiario en lista FATF High-Risk jurisdictions	Enhanced Due Diligence automático. Bloquear si país en lista negra FATF.
Dormant Account Activation	Cuenta inactiva por >12 meses que de repente recibe transacciones de alto monto	Cuenta sin movimiento >365 días con transacción > $5,000	Alerta + revisión de beneficiario. CDD refreshing obligatorio.

SAR Pipeline — Suspicious Activity Report

Paso	Actor	Acción	SLA
1	TMS (automático)	Genera alerta con evidencia: transacciones vinculadas, score ML, regla disparada, historial del cliente	Tiempo real (<30s)
2	Analista AML (Case Management)	Revisa el caso. Descarta (false positive) o confirma como actividad sospechosa	Máx. 24h desde alerta
3	Supervisor AML	Aprobación del SAR antes de filing	Máx. 48h desde alerta
4	Sistema de Reporte (automatizado)	Genera y envía SAR en formato XML regulatorio a la Unidad de Inteligencia Financiera (UIF/FIU) local	Dentro de plazo regulatorio (varía: 15–30 días)
5	Sistema	Registra SAR como inmutable en Kafka (bank.compliance.sar.filed) + audit log en Azure Log Analytics	Inmediato al filing
6	Customer Service (si aplica)	Restricción de cuenta según decisión del Supervisor — sin notificar al cliente (tipping-off prohibition)	Inmediato tras aprobación

CTR — Currency Transaction Report

Reporte obligatorio para transacciones en efectivo que superen el umbral regulatorio (varía por jurisdicción; referencia común: USD 10,000 / equivalente). El flujo es automático:

- Account Service detecta transacción de efectivo (tipo: CASH_DEPOSIT / CASH_WITHDRAWAL) que supera umbral.
- Publica evento bank.compliance.ctr.trigger en Kafka.
- Reporting Service consume el evento, genera el CTR en formato regulatorio y lo envía a la UIF dentro de las 15 días hábiles requeridos.
- El CTR queda registrado en el Case Management como caso cerrado con estado CTR_FILED.

Customer Due Diligence (CDD) y Niveles de Riesgo

Nivel	Perfil de riesgo	Controles CDD	Revisión periódica
Simplificado (SDD)	Cliente retail, transacciones bajas, sector no sensible, país bajo riesgo	Identificación básica (documento + selfie). Validación base de datos pública.	Cada 5 años
Estándar (CDD)	Perfil de riesgo medio. Cliente con historial normal.	ID verificación + prueba de domicilio + origen de fondos declarado. Screening PEP + Sanctions.	Cada 2 años
Reforzado (EDD)	PEP, cliente de país alto riesgo FATF, transacciones >$50K, persona jurídica compleja	CDD + prueba documental de origen de fondos + visita / entrevista + aprobación directiva. Beneficial Ownership hasta 25% de participación.	Cada 6 meses

Sanctions Screening — Listas de Verificación

El screening de sanciones se ejecuta en **dos momentos**:

- onboarding del cliente (Customer Service)
- en cada transacción internacional (Payment Service → SWIFT). Latencia objetivo: <200ms para no bloquear el flujo de pago.

Lista	Organismo	Alcance
SDN List (Specially Designated Nationals)	OFAC — USA	Personas y entidades con activos bloqueados por EE.UU.
UN Consolidated Sanctions List	Naciones Unidas	Lista global — Al-Qaeda, ISIS, proliferación nuclear
EU Consolidated Sanctions List	Unión Europea	Aplicable a operaciones con contrapartes EU
UK OFSI Consolidated List	HM Treasury — Reino Unido	Post-Brexit, lista independiente de la EU
Lista Local (UIF/UAFE/SBS)	Regulador local del país de operación	Personas y entidades sancionadas localmente
PEP Lists	WorldCheck / Refinitiv / Dow Jones	Personas Expuestas Políticamente — trigger de EDD automático
Adverse Media / Negative News	WorldCheck / ACAMS / fuentes abiertas	Noticias negativas vinculadas al cliente — señal de riesgo reputacional

7. Seguridad, IAM y Protección de Datos Personales

Actor	Mecanismo	Detalle
Cliente (web/móvil)	OAuth 2.0 + OIDC (Azure AD B2C)	Login usuario+contraseña + MFA obligatorio (TOTP/SMS). App móvil: biometría via FIDO2/WebAuthn. Access token JWT 15 min; Refresh token 8h rotativo.
Proveedor Open Finance	FAPI 1.0 Advanced + OAuth 2.0 + PKCE + PAR	Client certificate (mTLS) requerido. Scopes granulares por recurso y por usuario. Consentimiento explícito registrado como evento.
Empleado del banco	Azure AD + Conditional Access + MFA	Acceso solo desde dispositivos gestionados (MDM). Conditional Access bloquea acceso desde IPs no corporativas.
Servicio a Servicio (AKS interno)	mTLS + JWT de servicio (15 min)	Cada microservicio tiene certificado propio (gestionado por Istio/cert-manager). JWT de servicio firmado por Azure AD, validado en APIM Interno.
ACL Adapter → Core Bancario Tradicional	Service Account + SSL/TLS sobre IBM MQ · Credenciales en Azure Key Vault	El Core Legacy no soporta OAuth 2.0 ni mTLS nativo. Se usa una **Service Account dedicada** (usuario técnico sin acceso interactivo) con contraseña rotada automáticamente cada 30 días por Azure Key Vault.
ACL Adapter → Core Bancario Digital	mTLS + JWT de servicio (15 min)	El Core Digital es cloud-native y expone API REST. Mismo mecanismo que servicio a servicio: certificado gestionado por Istio, JWT de 15 min firmado por Azure AD.

Flujo de Inyección de Credenciales — ACL Adapter Legacy

```
// El pod del ACL Adapter Legacy NUNCA tiene credenciales en su imagen ni en su
código.
  // Las credenciales llegan en tiempo de ejecución vía CSI Secrets Driver.

  // 1. Kubernetes SecretProviderClass (definición declarativa):
  apiVersion: secrets-store.csi.x-k8s.io/v1
  kind: SecretProviderClass
  metadata:
    name: legacy-core-credentials
    namespace: acl-legacy            // namespace aislado — solo este namespace
accede
  spec:
    provider: azure
    parameters:
      keyvaultName: "kv-bancodem-prod"
      objects: |
        - objectName: "legacy-core-service-account-password"
          objectType: secret
          objectVersion: ""          // siempre obtiene la versión vigente
        - objectName: "legacy-core-mq-ssl-cert"
          objectType: cert
  ---
  // 2. El pod monta las credenciales como volumen (no como env var)
  // Env vars pueden filtrarse en logs — el volumen montado es más seguro.
  volumeMounts:
    - name: legacy-secrets
      mountPath: "/mnt/secrets/legacy"
      readOnly: true

  // 3. Azure Key Vault rota la contraseña cada 30 días automáticamente.
  //    Al rotar, el CSI Driver refresca el volumen sin reiniciar el pod.
  //    El ACL Adapter implementa un watcher que detecta el cambio y
  //    reconecta el IBM MQ Connection Factory con la nueva credencial.
```

Modelo de Autorización

- **RBAC para empleados:** roles teller, advisor, supervisor, auditor, admin. Cada rol tiene un conjunto de scopes permitidos. Los roles se asignan en Azure AD y se incluyen en el JWT.
- **ABAC para clientes:** el token JWT incluye claims como accountOwner:[ACC-123], consentScope:[accounts:read]. El APIM valida que el cliente solo acceda a sus propias cuentas.
- **Scopes Open Finance:** accounts:read, transactions:read, payments:write, statements:read. Cada scope requiere consentimiento explícito del cliente propietario.

Controles de Seguridad por Capa

Capa	Control	Tecnología
Perímetro (APIM Externo)	WAF, DDoS Protection, Rate Limiting, TLS 1.3, OAuth 2.0	Azure APIM + Azure Front Door + DDoS Standard
API Gateway Interno	mTLS, validación JWT, service identity, política de autorización	Azure APIM + Istio Service Mesh
Microservicio	Validación de input, RBAC/ABAC por endpoint, audit log por operación	Spring Security / OPA Gatekeeper
Datos en tránsito	TLS 1.3 obligatorio en todas las conexiones	Cert-manager (Kubernetes) + Azure Key Vault
Datos en reposo	Cifrado AES-256. Datos PII con campo-level encryption adicional	Azure Disk Encryption + TDE (SQL) + Cosmos DB encryption
Gestión de secretos	Rotación automática de claves y certificados	Azure Key Vault + CSI Secrets Driver en AKS
Red interna	Microsegmentación, Private Endpoints, no tráfico público entre servicios	Azure VNet + NSG + Private Link
Contenedores	Imágenes firmadas, escaneo de vulnerabilidades en CI/CD, políticas de pod security	Trivy + Azure Container Registry + OPA Gatekeeper

Protección de Datos Personales (LOPD / GDPR)

La arquitectura implementa privacidad por diseño (*Privacy by Design*) en cada capa, no como control adicional post-facto.

- **Minimización de datos:** cada microservicio almacena únicamente los atributos necesarios para su dominio BIAN. El Customer Service es el único propietario de datos de identificación personal.
- **Consentimiento explícito:** registrado como evento inmutable customer.consent.granted en Kafka, con timestamp, versión de la política y alcance del consentimiento.
- **Derecho al olvido:** se anonimiza el dato personal en el Customer Service (el identificador se reemplaza por un hash). Los eventos históricos en Kafka conservan el customerId pero el dato personal referenciado queda inaccesible. Event Sourcing compatible.
- **Data Residency:** todos los datos de clientes se almacenan en la región Azure local (ej. Brazil South). El replication DR usa Azure Paired Regions dentro del mismo país o región regulatoria.

- **Auditoría de acceso a PII:** toda operación sobre un dato personal genera un log de auditoría con: actor, servicio, operación, campo accedido, timestamp. Almacenado en Azure Log Analytics con retención de 5 años.
- **Cifrado campo-nivel para PII crítico:** IBAN, número de documento y datos biométricos se cifran a nivel de campo con clave propia del Customer Service, separada de la clave de cifrado del disco.

PCI DSS 4.0 — Cardholder Data Environment (CDE)

Alcance

PCI DSS 4.0 aplica a todo sistema que almacene, procese o transmita datos de tarjetas de pago (CHD — Cardholder Data). El principio rector de Banco DeMuestra es **minimizar el alcance del CDE al máximo**: mediante tokenización, ningún microservicio moderno toca el PAN (Primary Account Number) en claro. Solo el Payment Service dentro de la zona CDE procesa CHD durante el tiempo mínimo necesario para la autorización.

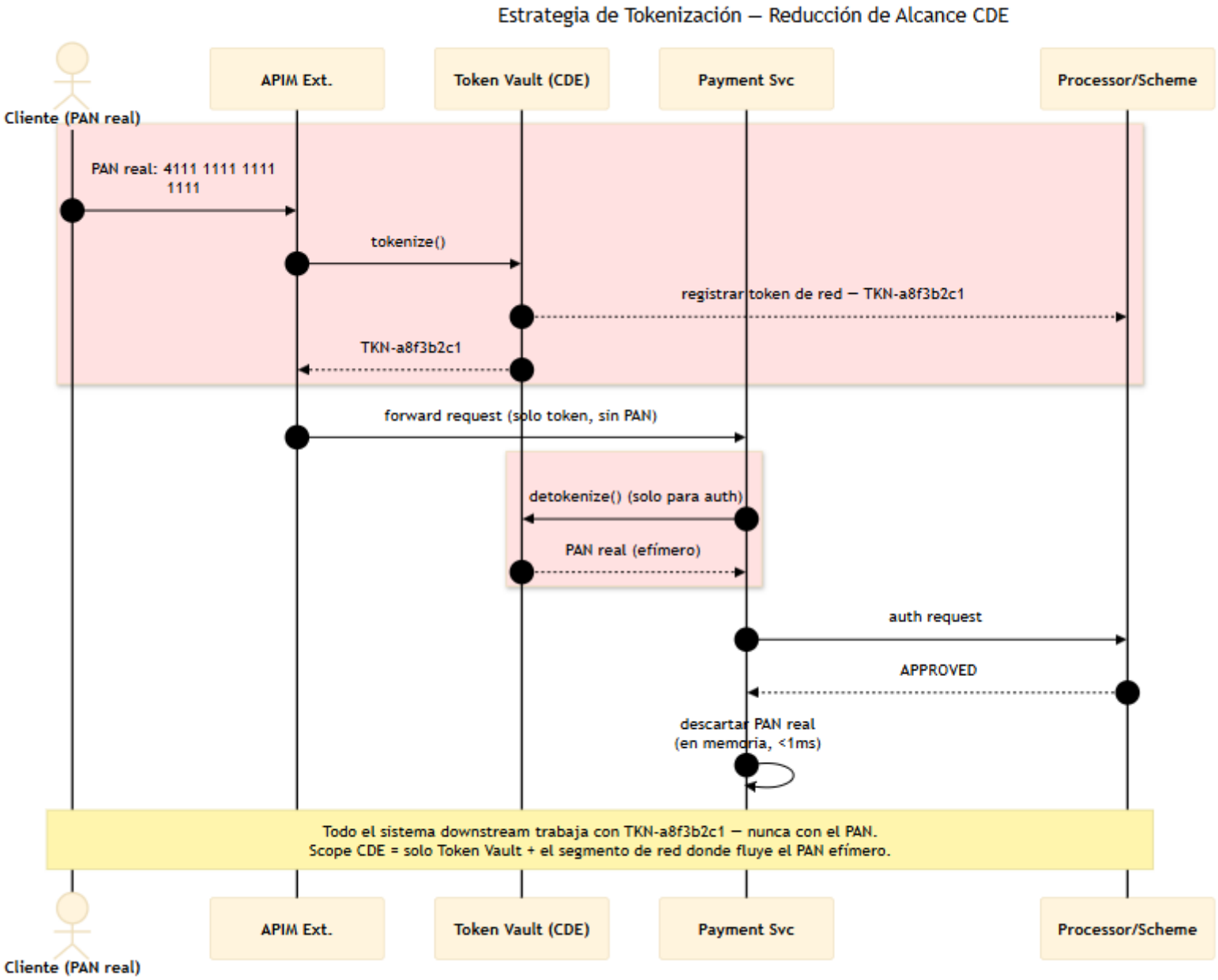
Datos en Alcance PCI DSS — CHD vs SAD

Dato	**Tipo**	**¿Permitido almacenar?**	**¿Requiere cifrado?**	**Tratamiento en Banco DeMuestra**
PAN (número de tarjeta)	CHD	Sí, pero solo con cifrado fuerte	Sí	**No se almacena en claro.** Se tokeniza en el punto de entrada.
Nombre del titular	CHD	Sí	Recomendado	Almacenado en Customer Service cifrado a nivel de campo (PII).
Fecha de expiración	CHD	Sí	Recomendado	Almacenada cifrada junto al token en Token Vault.
CVV2 / CVC2 (código de seguridad)	SAD	**NUNCA** — prohibición absoluta PCI DSS Req. 3.2	N/A	Usado solo en la validación online. Descartado inmediatamente. Nunca persiste en ningún storage.
PIN / PIN Block	SAD	**NUNCA** — prohibición absoluta	N/A	Solo en flujo ATM/POS cifrado con clave HSM. Nunca cruza los microservicios.
Track 1 / Track 2 (banda magnética)	SAD	**NUNCA**	N/A	Rechazado en el punto de entrada. Si llega, es bloqueado en WAF y logueado como incidente.

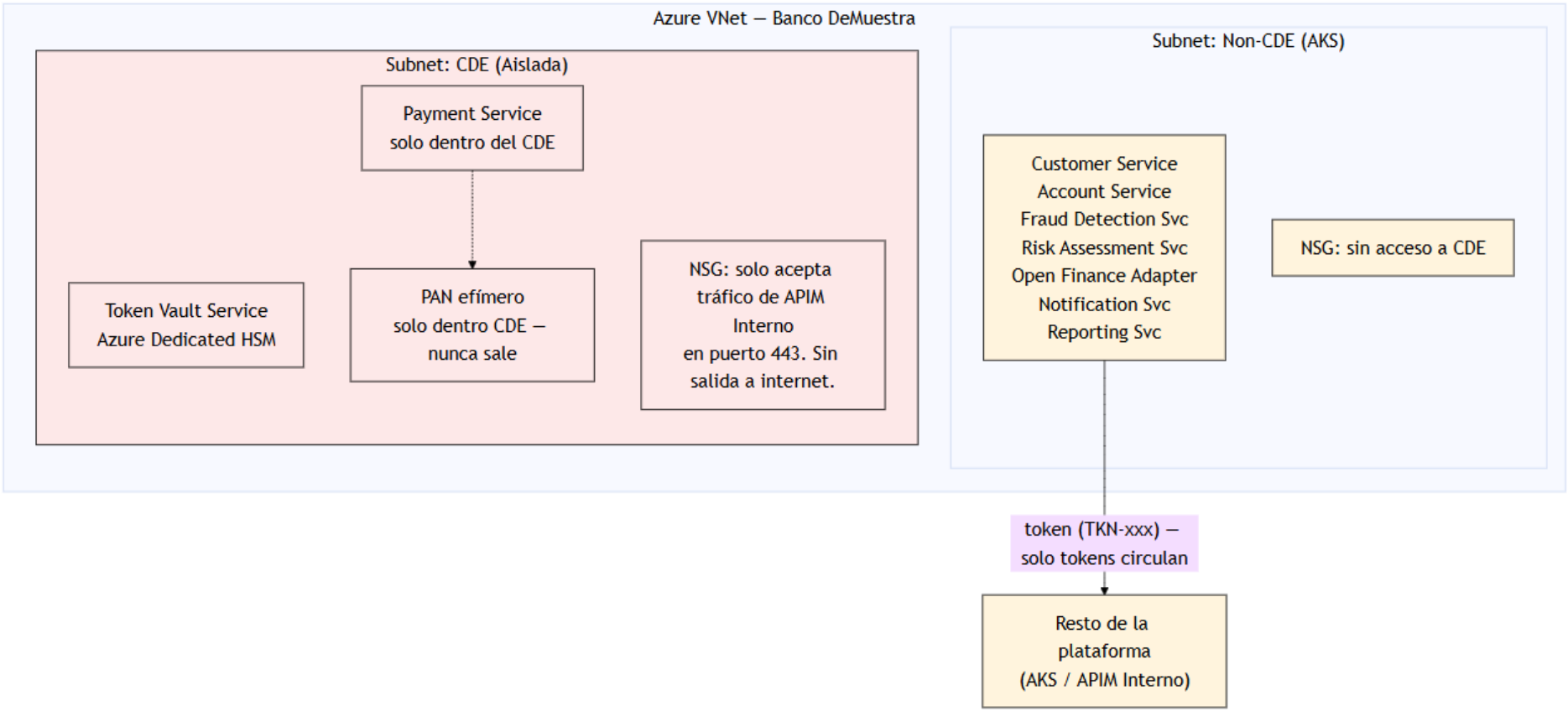
Estrategia de Tokenización — Reducción de Alcance CDE

La tokenización es el control más efectivo de PCI DSS:

- si un microservicio nunca ve el PAN real, no está en alcance del CDE. Banco DeMuestra usa un Token Vault centralizado que reemplaza el PAN por un token opaco en el punto de entrada, antes de que ningún microservicio lo procese.



Segmentación de Red del CDE en Azure



Gestión de Claves de Cifrado (PCI DSS Req. 3.5 / 3.6)

Clave	Uso	Almacenamiento	Rotación
DEK (Data Encryption Key)	Cifra el PAN en el Token Vault	Azure Dedicated HSM (FIPS 140-2 Level 3)	Anual
KEK (Key Encryption Key)	Cifra la DEK en reposo (envelope encryption)	Azure Key Vault Premium (HSM-backed)	Cada 2 años
MACsec Key	Cifra el enlace ExpressRoute (capa 2)	Gestionada por Azure / hardware de red	Automática (90 días)
TLS Certificate	Cifra tráfico en tránsito (todas las capas)	Azure Key Vault + cert-manager (AKS)	Automática (90 días)
Code Signing Key	Firma imágenes de contenedores y artefactos	Azure Key Vault	Anual

Controles PCI DSS 4.0 — Cumplimiento por Requisito

Requisito PCI DSS 4.0	Control implementado
Req. 1 — Firewall / NSG	Azure NSG con reglas por subnet. CDE subnet: ingress solo desde APIM Interno en puerto 443. Egress bloqueado a internet.
Req. 2 — No defaults	Hardening de imágenes base (CIS Benchmark). No contraseñas por defecto. Azure Policy audita configuraciones inseguras en AKS.
Req. 3 — Protección de CHD	PAN nunca almacenado en claro. Token Vault con AES-256-GCM + HSM. CVV2/SAD nunca almacenado (bloqueado en código + audit).
Req. 4 — Cifrado en tránsito	TLS 1.3 obligatorio. TLS 1.0/1.1 deshabilitado en Azure APIM y en todos los NSG. mTLS entre microservicios (Istio).
Req. 6 — Software seguro	SAST (SonarQube) + DAST (OWASP ZAP) en CI/CD. Trivy para vulnerabilidades de imágenes. SBOM generado en cada build.
Req. 7 — Acceso mínimo	RBAC en AKS (role binding por namespace). Solo el Payment Service tiene acceso al namespace CDE. Principio de least privilege.
Req. 8 — Autenticación fuerte	MFA obligatorio para todos los accesos a sistemas CDE. Sin acceso con usuario/contraseña solamente. Azure AD Conditional Access.
Req. 10 — Logs de auditoría	Todo acceso a CHD genera log inmutable en Azure Log Analytics. Retención 12 meses online, 36 meses cold. Alertas en tiempo real.
Req. 11 — Tests de seguridad	Escaneo ASV trimestral (Qualys/Rapid7). Penetration test anual por QSA. Vulnerability scanning interno semanal (Trivy + Defender for Cloud).
Req. 12 — Políticas	Política de seguridad de información documentada. Programa de concienciación anual. Inventario de sistemas CDE en alcance actualizado.

PAN Masking en Logs y APIs

```
// Regla: NUNCA el PAN completo aparece en logs, respuestas de API o mensajes de error
  // Implementación: interceptor en Payment Service (AOP / Spring aspect)

  // PAN real (solo existe efímeramente en memoria dentro del CDE):
  "4111111111111111"

  // PAN enmascarado en cualquier log o respuesta de API (Req. 3.3 PCI DSS):
  "411111******1111"   // primeros 6 + últimos 4 — BIN visible + identificación

  // En eventos Kafka (nunca circula el PAN — solo el token):
  { "cardToken": "TKN-a8f3b2c1-9f4e-4b2a-8c3d-1a2b3c4d5e6f" }

  // Filtro automático en logging framework (Logback / SLF4J):
  // Regex: \b(?:4[0-9]{12}(?:[0-9]{3})?|5[1-5][0-9]{14})\b → reemplaza por [PAN-MASKED]
```

8. Alta Disponibilidad y Recuperación ante Desastres

Alta Disponibilidad (Intra-Región)

Componente	Estrategia HA
Microservicios (AKS)	Mínimo 3 réplicas por servicio crítico, distribuidas en 3 Availability Zones. HPA (Horizontal Pod Autoscaler) basado en CPU + métricas de Kafka lag.
Apache Kafka	3 brokers en 3 AZ. Replication factor=3, min.insync.replicas=2. Controller redundante (KRaft mode).
Azure SQL (Account, Payment)	Always-On Availability Groups con réplica secundaria síncrona en AZ distinta. Failover automático en <30s.
Azure Cosmos DB (Customer, Fraud)	Multi-region writes habilitado. Consistencia Session para operaciones normales; Strong para operaciones críticas.
Azure APIM	Premium tier con unidades en múltiples AZ. Azure Front Door como balanceador global con health checks cada 5s.
Redis Cache	Azure Cache for Redis en Cluster Mode, 3 shards, 1 réplica por shard en AZ distinta.

Recuperación ante Desastres (Inter-Región)

Estrategia: Active-Passive con failover automático

La región primaria atiende todo el tráfico. La región secundaria está caliente (warm standby) y sincronizada continuamente.

Componente	Replicación DR
Kafka	MirrorMaker 2 replica todos los topics de primaria a secundaria con lag <5s.
Azure SQL	Active Geo-Replication asíncrona a región secundaria. RPO ~5s.
Cosmos DB	Replicación automática multi-región. Failover automático configurable.
AKS	Cluster espejo en región secundaria desplegado con misma configuración (IaC con Terraform). Imágenes pre-cargadas en ACR secundario.
DNS / Routing	Azure Traffic Manager con perfil de failover. Health checks cada 10s. Redirige tráfico a región secundaria en <1 min ante fallo.

Chaos Engineering

Se ejecutan pruebas trimestrales de DR con simulación de fallo de región completa usando Azure Chaos Studio. Los runbooks de failover están documentados y automatizados con Azure Automation Runbooks. El RTO objetivo se valida en cada prueba.

9. Estrategia de Integración Multicore

El banco opera simultáneamente con dos cores bancarios durante el período de transición. Esta coexistencia requiere una estrategia explícita para evitar inconsistencias, duplicación de datos y acoplamiento.

Principios del Multicore

- **Una sola fuente de verdad por entidad:** una cuenta o cliente pertenece a un único core en un momento dado. El metadato coreType: "LEGACY" | "DIGITAL" define el propietario.
- **Routing dinámico en Account Service:** antes de cualquier operación, el Account Service consulta el metadato de la cuenta y enruta al ACL Adapter correspondiente. El consumidor (Payment Service, etc.) nunca sabe a qué core habla.
- **ACL por core:** cada core tiene su propio ACL Adapter que traduce independientemente el modelo BIAN al modelo interno. Ningún servicio moderno conoce las estructuras internas de los cores.
- **Sincronización eventual vía eventos:** cambios críticos de estado (migración de cliente, cierre de cuenta) se publican en Kafka y ambos ACL Adapters escuchan para mantener coherencia.

Flujo de Routing Multicore

```
// Dentro del Account Service — método resolveCore()
interface CoreBankingPort {
    AccountBalance getBalance(String accountId);
    void executeDebit(String accountId, Money amount, String correlationId);
}

// El servicio resuelve el core en tiempo de ejecución
CoreBankingPort adapter = coreRegistry.resolve(account.getCoreType());
adapter.executeDebit(accountId, amount, correlationId);

// coreRegistry retorna:
//   coreType == "LEGACY"  → LegacyCoreAdapter  (via ACL Legacy)
//   coreType == "DIGITAL" → DigitalCoreAdapter  (via ACL Digital
```

Sincronización entre Cores

Evento	Acción en Legacy Core	Acción en Digital Core
customer.migrated	Marcar cliente como MIGRADO (solo lectura)	Crear registro completo del cliente
account.migrated	Marcar cuenta como MIGRADA; congelar escrituras	Crear cuenta con saldo sincronizado
payment.completed	ACL Legacy recibe evento; actualiza saldo si cuenta es LEGACY	ACL Digital recibe evento; actualiza saldo si cuenta es DIGITAL

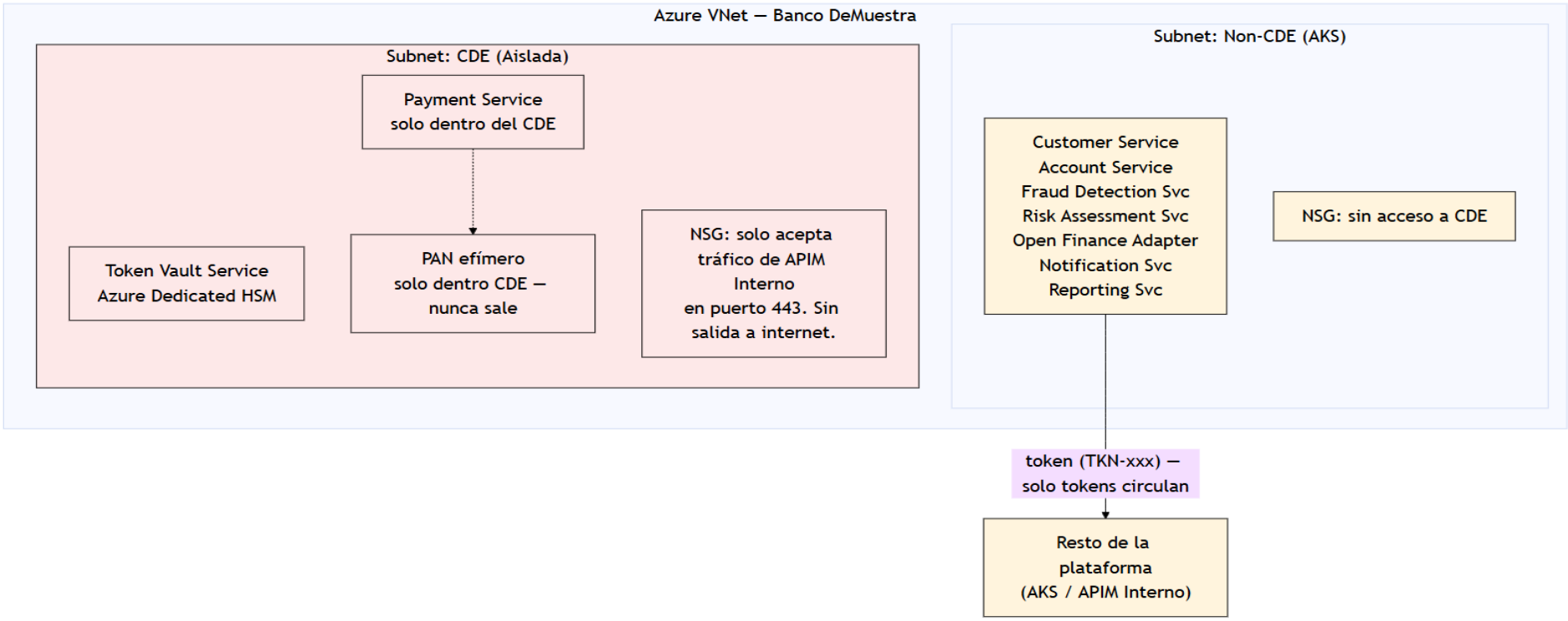
Garantía de consistencia

Durante la ventana de migración de una cuenta (típicamente <5 minutos), se bloquean escrituras en ambos cores mediante un distributed lock en Redis (clave: migration:lock:{accountId}). Solo se libera el lock cuando ambos ACL Adapters confirman la sincronización exitosa.

Conectividad con el Core Bancario Tradicional

El ACL Adapter Legacy descrito en la Sección 9 requiere una capa de conectividad física y de protocolo explícita. Los cores bancarios tradicionales raramente exponen APIs REST modernas, operan vía IBM MQ, ISO 8583, SOAP/XML, archivos batch o protocolos propietarios. Esta sección define cómo AKS en Azure alcanza esos sistemas con conectividad segura, resiliente y auditable.

Topología de Red — Datacenter On-Premise ↔ Azure



Azure ExpressRoute — Conectividad Dedicada

Parámetro	Configuración	Justificación
Tipo de circuito	ExpressRoute Direct 10 Gbps	Latencia <5ms datacenter ↔ Azure. Sin tráfico por internet público. SLA 99.95% de Microsoft.
Redundancia	Dos circuitos ExpressRoute en rutas físicas distintas (dual homing)	Elimina el circuito como SPOF. Failover automático si un circuito cae.
VPN Site-to-Site (backup)	Azure VPN Gateway (IPSec/IKEv2) como failover de ExpressRoute	Si ambos circuitos ExpressRoute fallan, el tráfico crítico cae automáticamente a VPN. Menor throughput pero garantiza continuidad.
Private Peering	Solo BGP Private Peering habilitado — sin Microsoft Peering público	El tráfico legacy nunca sale a internet. Todo permanece en la red privada de Microsoft.
Cifrado	MACsec en capa 2 (ExpressRoute Direct) + TLS 1.3 en capa aplicación	Doble cifrado: capa de enlace + capa de transporte.

Protocolos Legacy Soportados por el ACL Adapter

Protocolo Legacy	Caso de uso	Tecnología de adaptación	Traducción a
IBM MQ (MQSeries)	Operaciones transaccionales en tiempo real: débito, crédito, consulta de saldo	Spring Integration + IBM MQ client (JMS). ACL traduce mensaje MQ propietario → DTO BIAN.	REST JSON / Kafka evento
ISO 8583	Mensajería de autorización de tarjetas (POS, ATM, e-commerce)	jPOS framework (Java). Parser de bitmap ISO 8583 → modelo canónico BIAN PaymentExecution.	REST JSON / Kafka evento
SOAP / WSDL	Servicios de cliente y cuenta expuestos por el core como Web Services	Spring-WS client. Genera stub desde WSDL. Mapeo XSD → DTO BIAN en ACL.	REST JSON
Batch SFTP (COB files)	Cierre del día (Close of Business): archivos CSV/fixed-width con posición de cuentas, intereses, provisiones	Azure Data Factory pipeline. SFTP pull nocturno → transformación → carga en Reporting DB.	Azure SQL / Data Lake
Base de datos directa (JDBC)	Consultas de solo lectura a tablas del core (reporting, auditoría histórica)	Read-only JDBC connection pool vía Private Endpoint. Solo SELECT, nunca escritura directa.	REST JSON

Resiliencia de la Conexión Legacy

El Core Tradicional es el componente con menor SLA de toda la arquitectura. Los microservicios modernos deben estar diseñados para degradar graciosamente cuando el core legacy no responde, no para fallar en cascada.

Patrón	Implementación	Comportamiento ante fallo del Core Legacy
Circuit Breaker	Resilience4j en ACL Adapter Legacy. Umbral: 5 fallos en 10s → OPEN.	Estado OPEN: las llamadas fallan inmediatamente (fast-fail) sin esperar timeout.
Bulkhead	Thread pool dedicado para llamadas al core legacy (máx. 20 threads).	Si el core legacy satura su pool, no afecta a operaciones modernas
Timeout explícito	Timeout de 3s para consultas; 8s para operaciones transaccionales vía MQ.	Evita que un core lento bloquee threads indefinidamente.
Caché de consultas	Saldo y posición de cuentas legacy en Redis (TTL 30s para cuentas activas).	Si el core cae, el canal digital puede mostrar el último saldo conocido con indicador visual de "dato provisional"
Async fallback para pagos	Pagos cuyo ACL no pudo confirmar con el core quedan en estado ***PENDING_LEGACY*** en Kafka	El cliente recibe confirmación provisional. Si el reintento falla tras 4h → alerta operaciones para intervención manual.

Monitoreo de la Conexión Legacy

Métrica	Herramienta	Umbral de alerta
Estado Circuit Breaker ACL Legacy	Prometheus + Grafana (métrica Resilience4j)	Estado OPEN por >60s → alerta P1
Latencia IBM MQ round-trip	OpenTelemetry span + Application Insights	p99 >5s → alerta P2
Mensajes en cola MQ sin procesar	IBM MQ Console + Azure Monitor	>500 mensajes en queue → alerta P1
Throughput ExpressRoute	Azure Network Watcher	>80% de capacidad → alerta proactiva de capacidad
Pagos en estado PENDING_LEGACY	Custom metric Kafka + Grafana	>50 pagos pendientes >30min → alerta operaciones

10. Gobierno de APIs y Microservicios

Ciclo de Vida de APIs

Estado	Criterio de entrada	Responsable
DRAFT	Spec OpenAPI creada en repositorio; aún no publicada en APIM	Equipo propietario del servicio
REVIEW	Spec enviada al API Review Board; consumer-driven contract tests ejecutados	API Review Board
PUBLISHED	Aprobada por Review Board; desplegada en APIM; documentada en Developer Portal	DevOps / Release
DEPRECATED	Nueva versión publicada; aviso a consumers con 6 meses de antelación	Equipo propietario
RETIRED	Sin consumers activos; eliminada de APIM y Developer Portal	API Review Board

Gobierno de Microservicios

Dimensión	Decisión
Propiedad	Cada microservicio tiene un equipo propietario único (modelo "you build it, you run it").
Bounded Context	Ningún microservicio accede directamente a la base de datos de otro.
Service Mesh	Istio gestiona mTLS automático, load balancing, circuit breaking, retries y observabilidad de tráfico sin código en los servicios.
SLOs	Payment Service: disponibilidad 99.99%, latencia p99 <200ms. Fraud Detection: p99 <50ms. Account Service: 99.99%, p99 <100ms.
Dependency Rule	Las dependencias siempre apuntan hacia adentro (Clean Architecture).
Breaking Changes	Prohibido en la versión vigente.

11. Plan de Migración Gradual

Estrategia base

Strangler Fig + Feature Flags esto provoca que el API Gateway actúa como fachada permanente frente al legacy. El tráfico se redirige progresivamente por segmento de cliente usando feature flags en Azure App Configuration. Cada fase tiene rollback plan documentado y puede revertirse sin redeploy en <5 minutos.

Fase 0 **Meses 1–3**	**Fundación — Riesgo: CERO sobre legacy** Desplegar Azure APIM como fachada frente al core legacy (sin tocar el legacy). Instalar Kafka + AKS + Azure AD B2C + Key Vault + stack de observabilidad (Prometheus, Grafana, OpenTelemetry)
Fase 1 **Meses 4–6**	**Canales Digitales — Riesgo: BAJO** Lanzar Mobile App y Web Banking nuevos. El APIM enruta todas las requests al legacy vía ACL Adapter Legacy. Los clientes nuevos usan los nuevos canales pero el core sigue siendo el legacy.
Fase 2 **Meses 7–12**	**Core de Microservicios — Riesgo: MEDIO** Migrar primer segmento piloto: empleados del banco (100 cuentas internas). Validar paridad funcional durante 30 días.
Fase 3 **Meses 13–18**	**Pagos y Open Finance — Riesgo: MEDIO-ALTO** Strangler Fig activo: APIM decide por metadato de cuenta. Monitorear divergencia de saldos entre cores con reconciliación nocturna automatizada.
Fase 4 **Meses 19–24**	**Migración Masiva — Riesgo: CONTROLADO** Migrar 80% de clientes al Core Digital en oleadas semanales de 5,000 clientes. Legacy Core en modo read-only para cuentas migradas.
Fase 5 **Mes 25+**	**Decommission Legacy — Riesgo: BAJO** Descomisionar infraestructura legacy gradualmente. Mantener ACL Legacy como adaptador de consulta histórica hasta cumplir con los plazos de retención regulatoria (mínimo 7 años).

12. Observabilidad y Trazabilidad

Los Tres Pilares

Pilar	Herramienta	Qué mide
Métricas	Prometheus + Grafana + Azure Monitor	Negocio: pagos por segundo, tasa de éxito de pagos, alertas de fraude activas, clientes migrados.
Logs	ELK Stack / Azure Log Analytics	Logs estructurados JSON con campos obligatorios
Trazas distribuidas	OpenTelemetry + Azure Application Insights / Jaeger	Sampling: 100% para errores y operaciones financieras; 10% para consultas de saldo.

Correlation ID — Trazabilidad de Extremo a Extremo

El ***correlationId*** es generado por Azure APIM en el punto de entrada y propagado por toda la cadena:

```
// 1. APIM genera y adjunta en header HTTP
X-Correlation-ID: corr-550e8400-e29b-41d4-a716-446655440000

// 2. Cada microservicio lo propaga en llamadas salientes
RestTemplate headers: { "X-Correlation-ID": correlationId }
gRPC metadata:        { "x-correlation-id": correlationId }

// 3. Payment Event Publisher lo incluye en el envelope Kafka
{ "correlationId": "corr-550e8400-...", "eventType":
"payment.authorized", ... }

// 4. Todo log incluye el correlationId
{ "level":"INFO", "service":"payment-service",
"correlationId":"corr-550e8400-...",
  "message":"Payment authorized", "paymentId":"PAY-UUID" }
```

Con el ***correlationId*** es posible reconstruir en Jaeger/Application Insights toda la cadena de una transacción: desde el tap en la app del cliente hasta la confirmación en el core bancario, incluyendo la verificación de fraude y el evento en Kafka.

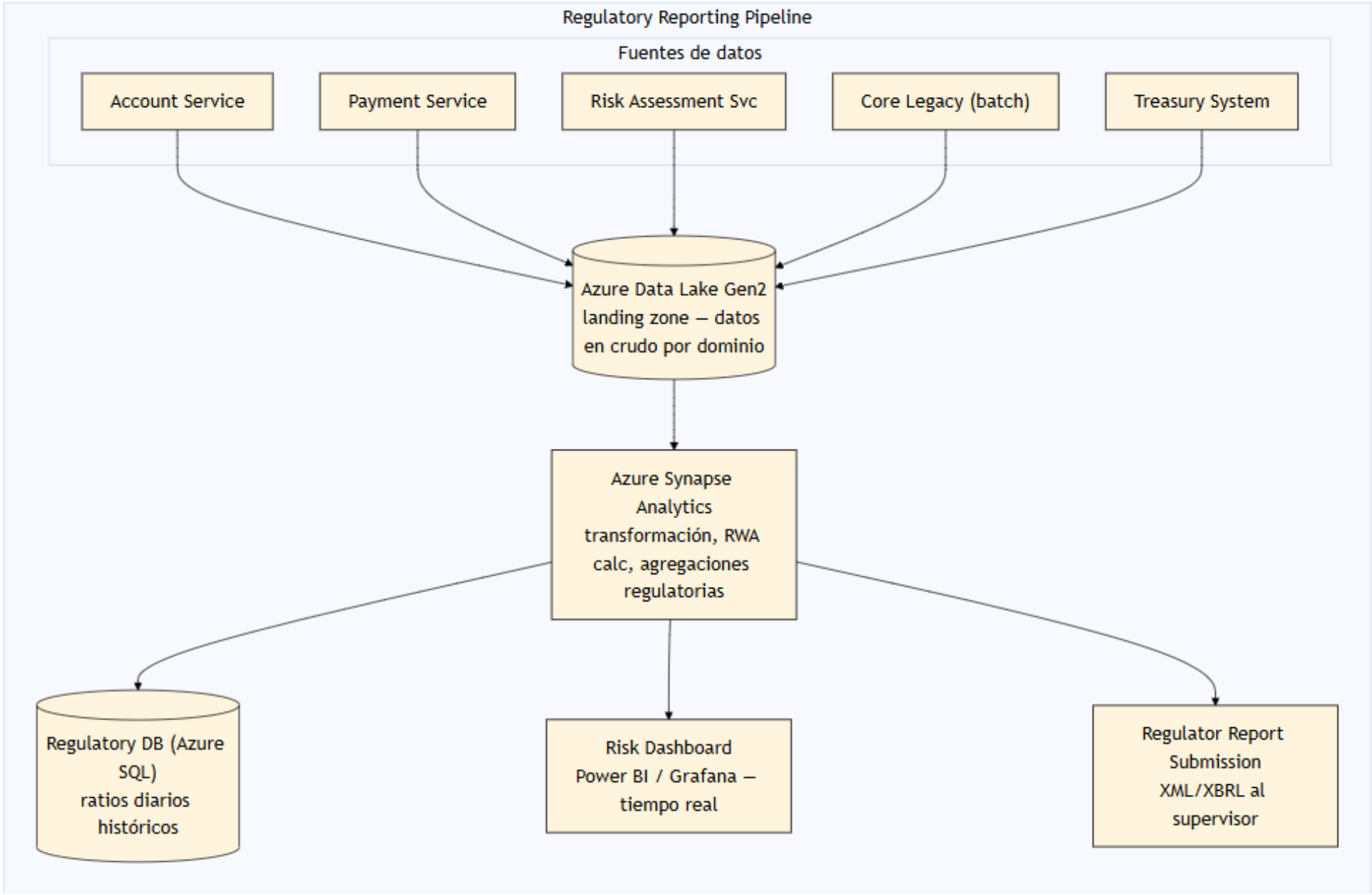
Alertas y SLO Monitoring

Alerta	Umbral	Acción
Error rate Payment Service	>0.1% en 5 min	PagerDuty → On-call. Revisar DLQ y Circuit Breaker.
Kafka consumer lag	>10,000 mensajes	Escalar consumers automáticamente (HPA)
Fraud alert spike	>50 alertas/min	Notificación inmediata al equipo de fraudes.
Latencia p99 > SLO	Payment >200ms por 5 min	Alerta a equipo. Revisar Circuit Breaker y dependencias lentas.
DLQ con mensajes	Cualquier mensaje en DLQ	Alerta inmediata a equipo operaciones para análisis y replay.

Métricas Regulatorias Clave y su Fuente de Datos

Métrica	Definición	Umbral mínimo regulatorio	Fuente de datos en la arquitectura
CET1 (Common Equity Tier 1)	Capital de mayor calidad (acciones ordinarias + reservas) como porcentaje de los Activos Ponderados por Riesgo (RWA)	≥ 4.5% + colchón de conservación 2.5% = **7% mínimo efectivo**	Risk Assessment Service consume posiciones de Account Service + Credit Risk + datos del Core (via ACL)
Tier 1 Capital Ratio	CET1 + AT1 (Additional Tier 1) / RWA	≥ 6%	Risk Assessment Service + datos de instrumentos de capital del Core
Total Capital Ratio	(Tier 1 + Tier 2) / RWA	≥ 8%	Risk Assessment Service
LCR (Liquidity Coverage Ratio)	HQLA (High Quality Liquid Assets) / Salidas netas de efectivo estimadas a 30 días	≥ 100%	Account Service (posición de caja) + Treasury System (HQLA portfolio)
NSFR (Net Stable Funding Ratio)	Financiación estable disponible / Financiación estable requerida	≥ 100%	Account Service (pasivos por vencimiento) + Treasury System
Leverage Ratio	Tier 1 Capital / Exposición total (on-balance + off-balance)	≥ 3% (≥ 3.5% para G-SIBs)	Risk Assessment Service + Account Service + datos de contingentes del Core
RWA (Risk-Weighted Assets)	Activos ajustados por su peso de riesgo crediticio, mercado y operacional	Denominador de todos los ratios de capital	Credit Risk Assessment Service — calcula RWA por exposición usando modelos SA o IRB

Arquitectura del Regulatory Reporting Pipeline



Cálculo de RWA — Enfoque Estándar (SA) vs Modelos Internos (IRB)

Enfoque	Descripción	Aplicado a	Implementación en Risk Svc
SA (Standardised Approach)	Ponderaciones de riesgo fijas definidas por el regulador según tipo de exposición	Cartera retail y PyME. Exposiciones de menor complejidad.	tabla de ponderaciones por categoría. Lookup table en Redis
F-IRB (Foundation IRB)	Banco calcula PD (Probability of Default) internamente.	Cartera corporativa y grandes empresas.	Credit Risk Assessment Service calcula PD por exposición usando modelos propios. LGD y EAD son parámetros fijos del regulador.
A-IRB (Advanced IRB)	Banco calcula PD, LGD y EAD internamente. Requiere aprobación supervisora explícita.	Futuro — Fase 4 de madurez del modelo de riesgo.	Requiere validación de modelos por auditoría independiente y aprobación del regulador. No en scope inicial.

Frecuencia de Cálculo y Reporte

Métrica	Frecuencia de cálculo	Frecuencia de reporte al regulador	Herramienta
CET1, Tier 1, Total Capital	Diaria (batch COB)	Trimestral (COREP)	Azure Synapse + XBRL renderer
LCR	Diaria (batch COB) + intraday estimado	Mensual	Azure Synapse
NSFR	Diaria (batch COB)	Trimestral	Azure Synapse
Leverage Ratio	Diaria	Trimestral (COREP)	Azure Synapse
RWA por cartera	Diaria (batch COB)	Trimestral	Azure Synapse
Large Exposures	Diaria — alertas en tiempo real si >10% capital	Trimestral	Risk Assessment Svc + Azure Monitor

Formato de Reporte — COREP / XBRL

Los reportes regulatorios se generan en formato **XBRL** (eXtensible Business Reporting Language), estándar exigido por la EBA (European Banking Authority) y la mayoría de reguladores bancarios para el reporte COREP (Common Reporting Framework):

- **Taxonomía:** EBA ITS on Supervisory Reporting — versión vigente. El pipeline de Synapse mapea los valores calculados a los conceptos XBRL de la taxonomía.
- **Validación:** El archivo XBRL pasa por un validador de reglas de negocio (EBA Validation Rules) antes del envío. Errores de validación bloquean el envío y generan alerta P1 al equipo de Regulatory Reporting.
- **Envío:** Transmisión segura al portal del regulador (ONTIC/CASANDRA o equivalente local) vía SFTP + firma digital. Confirmación de recepción almacenada en Azure Log Analytics con retención de 10 años.
- **Auditoría:** Cada submission queda en el Event Store de Reporting Service como evento inmutable (regulatory.report.submitted) con hash del archivo enviado.

Estrategia de Testing — Calidad y Seguridad en Todos los Niveles

El ejercicio enfatiza testability como principio de diseño. Esta estrategia no es un proceso posterior al desarrollo — está integrada en el diseño arquitectónico

Pirámide de Testing por Capa

Nivel	Tipo de test	Herramienta	Cobertura objetivo	Cuándo se ejecuta
Unit Tests	Lógica de dominio aislada. Sin dependencias externas — el DIP garantiza que el Domain Layer es testeable con mocks simples.	JUnit 5 + Mockito	>85% en capa Domain y Application	En cada commit (CI, <2 min)
Contract Tests	Verifica que productor y consumidor de una API o evento acuerdan el mismo contrato. Consumer-driven: el consumidor define sus expectativas, el productor las verifica.	Pact (REST APIs) + AsyncAPI Schema Registry (eventos Kafka)	100% de integraciones inter-servicio	En cada PR antes de merge
Integration Tests	Prueba el servicio completo con sus dependencias reales (DB, Kafka). Valida la integración entre capas sin mocks de infraestructura.	Testcontainers (Kafka, PostgreSQL, Redis en Docker) + Spring Boot Test	Flujos críticos: pago, fraud check, onboarding	En pipeline CI (entorno efímero)

Performance Tests	Valida que los SLOs de latencia y throughput se cumplen bajo carga realista, estrés y picos.	Gatling (carga sostenida) + k6 (scripting flexible y CI-friendly)	Ver perfiles de carga en 12C.2	Nightly en staging. Obligatorio antes de cada release mayor.
DAST (Dynamic Security)	Prueba de seguridad de APIs en ejecución — inyección, autenticación, headers, exposición de datos sensibles.	OWASP ZAP (modo API scan) en pipeline CI/CD	Todos los endpoints públicos de APIM Externo	En pipeline staging antes de prod
Penetration Test	Evaluación ofensiva de la arquitectura completa por un equipo externo especializado en fintech/banking.	Empresa externa certificada (CREST/OSCP). Scope: APIM, AKS, CDE, ExpressRoute.	Arquitectura completa incluyendo CDE (PCI DSS Req. 11)	Anual obligatorio (PCI DSS). Adicional tras cambios arquitectónicos mayores.
Chaos Engineering	Valida resiliencia ante fallos reales: caída de un AZ, latencia de Kafka, fallo del Core Legacy, timeout de Fraud Service.	Azure Chaos Studio	Todos los escenarios de DR documentados en Sección 8	Trimestral. Resultados validan RTO/RPO objetivo.

Perfiles de Carga — Performance Testing (Gatling / k6)

Perfil	TPS objetivo	Duración	SLO que valida	Criterio de fallo
Baseline (normal)	500 TPS pagos + 2,000 TPS consultas de saldo	30 min sostenido	Payment p99 <200ms; Balance p99 <100ms; error rate <0.1%	Cualquier SLO incumplido
Stress	2x baseline: 1,000 TPS pagos + 4,000 TPS consultas	15 min sostenido	Degradación aceptable: p99 <500ms; error rate <1%	Error rate >1% o p99 >500ms
Spike	Rampa 0→5,000 TPS en 30s (simulación campaña viral / Black Friday)	5 min pico + 10 min recuperación	HPA escala en <60s; recuperación a baseline en <120s	Pérdida de transacciones o saldo incorrecto post-spike
Soak	Baseline durante 8h continuas	8 horas	Sin memory leaks ni degradación progresiva de latencia	Latencia p99 crece >20% entre hora 1 y hora 8

Reconciliación Financiera — T+0, T+1 y Nostro Management

La reconciliación no es un proceso contable posterior — es un control de integridad del sistema en tiempo real. Un desfase entre lo que el Payment Service registró y lo que el Core Bancario refleja es un incidente operativo de alta prioridad, no una diferencia contable a resolver al mes siguiente.

Reconciliación Intraday (T+0) — Posición en Tiempo Real

Qué se reconcilia	Frecuencia	Fuentes comparadas	Tolerancia	Acción ante desfase
Posición de caja (cash position)	Cada 15 minutos durante el horario operativo	Payment Service Event Store vs. saldo en Core Bancario (via ACL Adapter)	Diferencia = 0 (cero tolerancia en saldos)	Alerta P1 inmediata. Suspender nuevos pagos hasta resolución. Escalar a CRO y Tesorería.
Pagos en tránsito (PENDING_LEGACY / IN_TRANSIT)	Cada 5 minutos	Payment Service (estado IN_TRANSIT) vs. confirmaciones SWIFT recibidas (pacs.002)	Pagos sin confirmación >30 min generan alerta	Polling automático de estado. Si >4h sin respuesta: enviar camt.056 (cancellation request).
Eventos Kafka vs. actualizaciones Core	Continuo — Kafka Streams	Eventos ***bank.payments.completed*** vs. movimientos registrados en Account Service read model	Lag <100ms (consistencia eventual esperada)	Si lag >5 min: alerta al equipo de operaciones. Posible projector detenido.

Reconciliación de Cierre (T+0 EOD — Close of Business)

Al cierre del día operativo, Azure Data Factory ejecuta el pipeline COB:

- **Extracción:** Payment Service exporta todos los movimientos del día en formato estructurado → Azure Data Lake.
- **Extracción Legacy:** SFTP pull del archivo batch del Core Bancario Tradicional (posiciones de cuenta, intereses, comisiones del día).
- **Matching:** Synapse Analytics cruza ambas fuentes por correlationId y amount. Genera reporte de matches y breaks.
- **Breaks:** Toda diferencia (break) genera un ticket automático en el sistema de Case Management con los detalles de la discrepancia.
- **SLA:** El reporte COB debe estar disponible antes de las 23:59 del mismo día. Si el pipeline falla, alerta P1 a Operaciones.

Reconciliación T+1 — Settlement y Nostro

Tipo	Descripción	Fuentes	Proceso
Settlement T+1	Verificación de que los pagos procesados ayer fueron efectivamente liquidados por el banco corresponsal	Mensajes camt.053 (Bank Statement) recibidos de SWIFT vs. pagos en estado COMPLETED en Payment Service	ISO 20022 Translation Service convierte camt.053 → modelo canónico → Reconciliation Service cruza por ***EndToEndId*** (campo ISO 20022). Breaks escalados a Correspondent Banking team.
Nostro Reconciliation	Reconciliación de la cuenta que Banco DeMuestra mantiene en el banco corresponsal (cuenta nostro) contra los registros internos de movimientos internacionales	Estado de cuenta nostro (camt.053) vs. libro de corresponsalía interno	Proceso diario. Diferencias >$1,000 = alerta. Diferencias >$10,000 = escalada inmediata a Tesorero. Proceso de investigación máx. 48h.
Aging de Breaks	Control de la antigüedad de diferencias no resueltas	Sistema de Case Management (breaks abiertos)	Breaks >3 días hábiles: escalada automática a supervisor. >5 días: escalada a CFO. Todo break queda en el registro de auditoría.

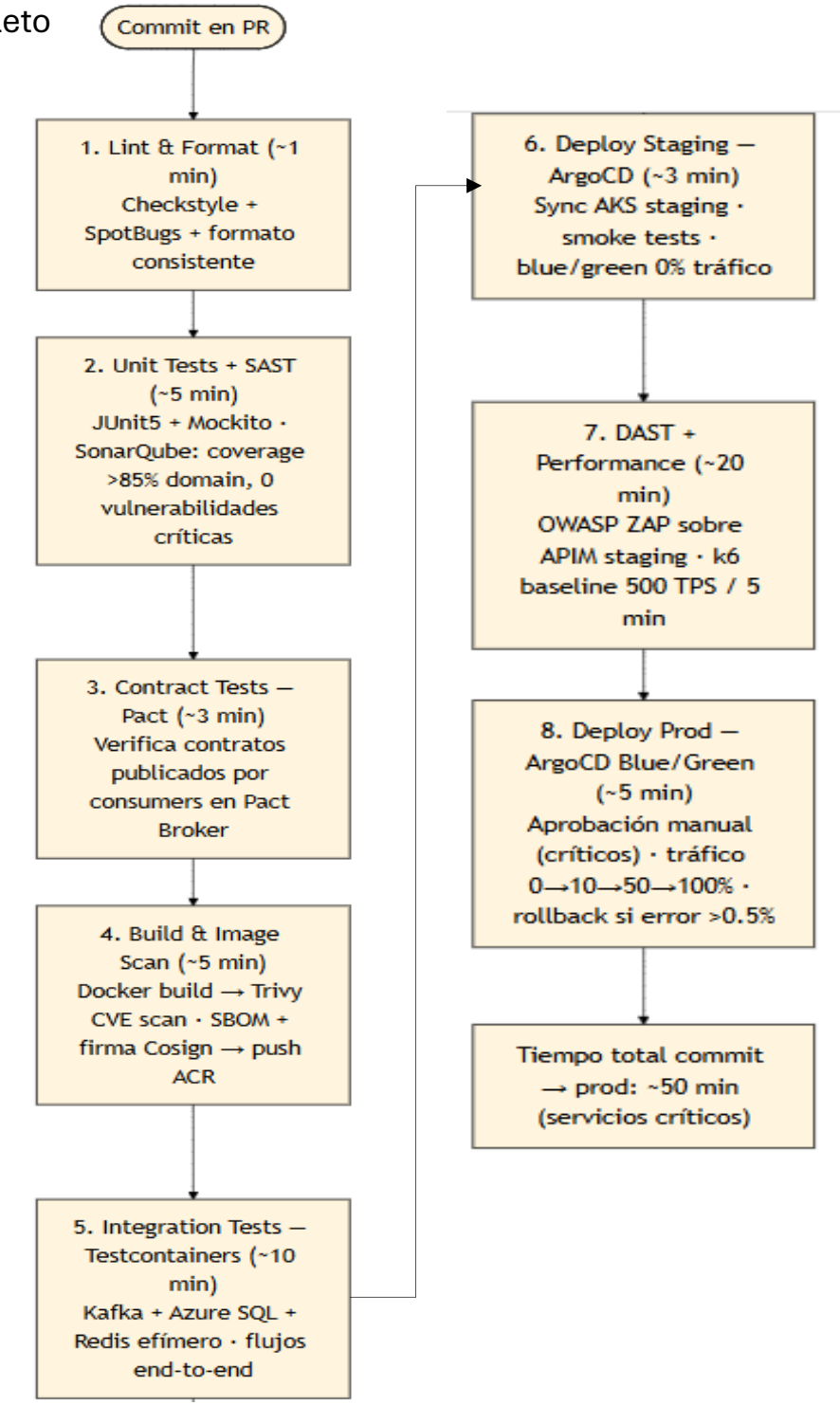
13. Pipeline CI/CD — DevSecOps

El pipeline CI/CD no es solo automatización de despliegue, es el mecanismo que garantiza que cada cambio cumple los estándares de calidad, seguridad y contrato antes de llegar a producción. Cada stage tiene criterios de fallo explícitos (quality gates) que bloquean el pipeline si no se cumplen.

Herramientas y Responsabilidades

Herramienta	Rol	Stage del pipeline
Azure DevOps Pipelines	Orquestador principal del pipeline CI/CD	Todos los stages
SonarQube	SAST: análisis estático de código (code smells, vulnerabilidades, coverage gate)	Build
Trivy	Escaneo de vulnerabilidades en imágenes Docker (CVEs) y dependencias (SBOM)	Build / Image scan
Pact Broker	Publicación y verificación de contratos consumer-driven entre microservicios	Contract test
Testcontainers	Infraestructura efímera para integration tests (Kafka, SQL, Redis en Docker)	Integration test
OWASP ZAP	DAST: escaneo de seguridad dinámico contra la API desplegada en staging	Security scan (staging)
Gatling / k6	Performance testing contra staging (perfil baseline mínimo en cada release)	Performance (staging)
ArgoCD	GitOps: sincronización declarativa de manifiestos Kubernetes en AKS	Deploy (staging + prod)
Cosign / Notation	Firma de imágenes de contenedor antes del push a Azure Container Registry	Image sign

Stages del Pipeline — Flujo Completo



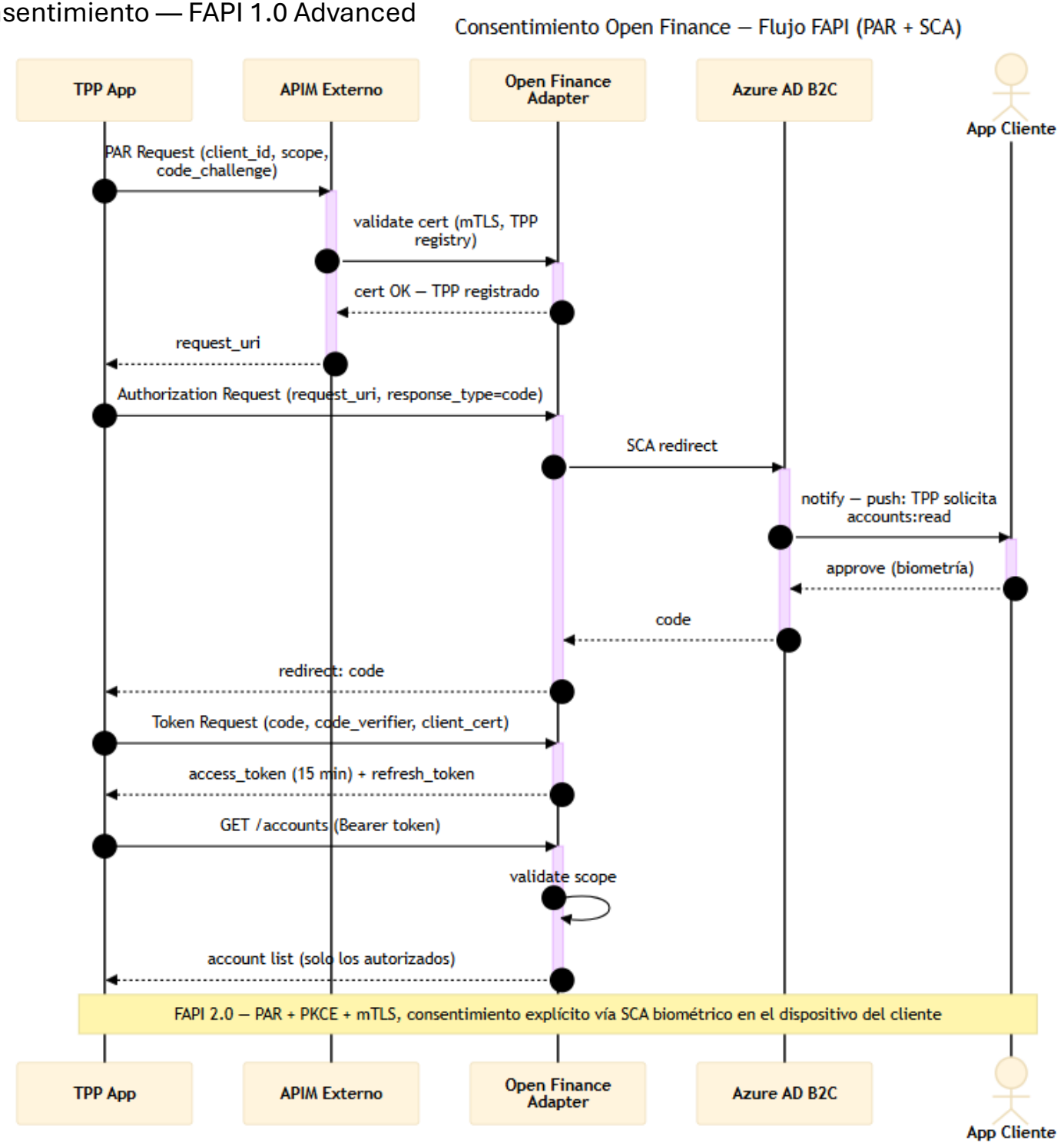
Open Finance — Framework Regulatorio y Diseño de Consentimiento

La arquitectura de Open Finance de Banco DeMuestra está diseñada para ser **agnóstica al framework regulatorio**: los principios de consentimiento, autenticación fuerte, y acceso granular por scope son comunes a todos los marcos vigentes (PSD2, Open Finance Brasil, FDX USA, UK Open Banking). La capa de adaptación regulatoria está encapsulada en el **Open Finance Adapter Service**, de modo que un cambio de regulación no afecta a los microservicios core.

Elementos Comunes a Todos los Frameworks de Open Finance

Elemento	Implementación en Banco DeMuestra
Registro de TPP (Third-Party Provider)	Los proveedores se registran en Azure APIM con certificado de cliente (mTLS). El Open Finance Adapter valida que el TPP está activo y en buen estado en el directorio de proveedores autorizado por el regulador local.
Autenticación Fuerte del Cliente (SCA)	FAPI 1.0 Advanced + OAuth 2.0 + PKCE + PAR. El cliente autoriza explícitamente a través de Azure AD B2C con MFA. Ningún TPP puede obtener acceso sin consentimiento activo del cliente.
Consentimiento Granular	El cliente aprueba cada scope individualmente. El consentimiento queda como evento inmutable en Kafka
Re-consentimiento periódico	Los tokens de larga duración expiran y requieren re-consentimiento del cliente (configurable según marco regulatorio: 90 días en PSD2, diferente en otros). El Token Management BQ del Open Finance Adapter gestiona el ciclo de vida.
Revocación de consentimiento	El cliente puede revocar el consentimiento en cualquier momento desde la app
Registro de accesos	Cada acceso de un TPP a datos del cliente genera un log de auditoría con: TPP ID, scope, recurso accedido, timestamp. El cliente puede ver el historial de accesos desde la app.
Sandbox / Developer Portal	Azure APIM Developer Portal expone las APIs de Open Finance con datos sintéticos. Los TPPs pueden desarrollar y probar sin tocar datos reales. Certificación requerida antes de acceso a producción.

Flujo de Consentimiento — FAPI 1.0 Advanced



Anexo A — Matriz de Dominios BIAN (Release v14.0.0)

Service Domain BIAN	Patrón Funcional	Microservicio	Action Terms aplicados	Core Propietario
Customer Management	Direct	Customer Service	Initiate, Update, Request, Retrieve, Exchange	Core Digital
Know Your Customer (KYC)	Assess	Customer Service	Initiate, Execute, Retrieve, Notify	Core Digital
Current Account	Direct	Account Service	Initiate, Control, Execute, Retrieve, Notify	Ambos (routing por coreType)
Savings Account	Direct	Account Service	Initiate, Control, Execute, Retrieve	Ambos (routing por coreType)
Payment Execution	Fulfill	Payment Service	Initiate, Execute, Update, Request, Retrieve, Notify	Core Digital
Credit Transfer	Fulfill	Payment Service (sub-función)	Initiate, Execute, Retrieve, Notify	Core Digital
Fraud Diagnosis	Monitor	Fraud Detection Service	Initiate, Control, Execute, Retrieve, Notify	Sistema propio (ML)
Credit Risk Assessment	Assess	Risk Assessment Service	Initiate, Execute, Retrieve, Notify	Sistema propio (Rules Engine)
Customer Agreement	Agree	Customer Service	Initiate, Request, Update, Retrieve, Exchange	Core Digital
Regulatory Reporting	Support	Reporting Service	Retrieve, Request, Notify	Ambos (consume eventos)
Open Finance Operations	Operate	Open Finance Adapter	Initiate, Execute, Request, Retrieve, Exchange	Core Digital

A.2 Control Records y Behavior Qualifiers por Servicio

Concepto BIAN: El *Control Record (CR)* es el objeto maestro (aggregate root) de cada Service Domain. Los *Behavior Qualifiers (BQ)* son sub-módulos del CR que representan aspectos específicos del dominio. Cada BQ puede ser operado independientemente con los Action Terms estándar. Patrón de URL: /{ServiceDomain}/{cr-id}/{BehaviorQualifier}/{bq-id}/{ActionTerm}

A.3 Coreless Banking 3.0 (BIAN)

La estrategia multicore descrita en la Sección 9 está alineada con el concepto **Coreless Banking 3.0** definido por BIAN. Este modelo propone desacoplar las capacidades bancarias de cualquier core monolítico, permitiendo que el banco seleccione proveedores best-of-breed por Service Domain sin problemas de interoperabilidad. El ACL Adapter actúa como el *BIAN Semantic Translator* que convierte el modelo propietario de cada core

al modelo canónico BIAN, garantizando que los microservicios modernos sean agnósticos al core subyacente.

Service Domain	Control Record (CR)	Behavior Qualifiers (BQ)	Microservicio implementador
Payment Execution	***PaymentExecutionProcedure***	**PaymentInstruction** — instrucción de transferencia **PaymentMechanism** — SWIFT / ACH / SEPA **FraudCheck** — resultado del check de fraude **RegulatoryCheck** — AML, OFAC screening	Payment Service
Current Account	***CurrentAccountFulfillmentArrangement***	**Balance** — saldo disponible y contable **Interest** — cálculo y acumulación de intereses **Restrictions** — bloqueos, embargos **Transactions** — historial de movimientos **Mandate** — poderes y firmas autorizadas	Account Service → ACL Adapter (Legacy / Digital)
Customer Management	***CustomerRelationshipManagementPlan***	**Identity** — datos de identificación (PII cifrado) **ContactDetails** — email, teléfono, dirección **DocumentDirectory** — documentos KYC **Consent** — consentimientos Open Finance / LOPD **Party** — estructura legal (persona natural / jurídica)	Customer Service
Fraud Diagnosis	***FraudDiagnosisAssessment***	**RuleSet** — modelos ML activos y versiones **AnalysisResult** — score y regla disparada **CustomerBehaviorModels** — perfil de comportamiento **Alert** — alerta generada y acción recomendada	Fraud Detection Service
Credit Risk Assessment	***CreditRiskAssessmentAssessment***	**AssetandLiabilityValuation** — valoración del cliente **RegulatoryRequirement** — basilea III, provisiones **CustomerCreditRatingDetails** — score crediticio interno **CreditBureauReport** — reporte del buró externo	Risk Assessment Service
Open Finance Operations	***OpenFinanceOperatingSession***	**ConsentAuthorization** — flujo de consentimiento FAPI **AccountAccessConsent** — scope accounts:read **PaymentConsent** — scope payments:write **TokenManagement** — access / refresh token lifecycle	Open Finance Adapter Service

Resultado: cuando el Core Tradicional se decomisione (Fase 5), los microservicios no requieren ningún cambio — solo se retira el ACL Adapter Legacy. La arquitectura es inherentemente portable entre proveedores de core bancario.

Anexo B — Matriz de Sistemas de Registro

Entidad	System of Record	Servicio Propietario	Otros sistemas que la referencian
Cliente (existente)	Core Bancario Tradicional	ACL Adapter Legacy → Legacy Core	Account Svc (via ACL), Fraud Svc
Cliente (nuevo / migrado)	Core Bancario Digital	Customer Service → Digital Core	Account Svc, Fraud Svc, Open Finance
Cuenta (legacy)	Core Bancario Tradicional	ACL Adapter Legacy	Payment Svc (via ACL), Reporting
Cuenta (digital)	Core Bancario Digital	Account Service	Payment Svc, Open Finance, Reporting
Saldo de cuenta	Core Bancario (según coreType)	Account Service (proyección CQRS read)	Web Banking, Mobile App (solo lectura)
Transacción de pago	Payment Service DB (Azure SQL)	Payment Service	Reporting Svc, Account Svc (eventos)
Evento de fraude	Fraud Detection Service DB (Cosmos DB)	Fraud Detection Service	Payment Svc (bloqueo), Risk Svc
Evaluación de riesgo	Risk Assessment Service DB (Azure SQL)	Risk Assessment Service	Payment Svc (decisión de autorización)
Consentimiento Open Finance	Customer Service DB	Customer Service	Open Finance Adapter (validación de scope)
Historial de eventos (audit)	Apache Kafka (inmutable) + Azure Log Analytics	Plataforma (compartido, solo lectura)	Reporting, Auditoría, DR replay

Anexo C — Tabla de Controles de Seguridad

Control	Categoría	Capa	Tecnología	Estándar
Autenticación de clientes	Preventivo	APIM Externo	Azure AD B2C + OAuth 2.0 + MFA	OIDC, FAPI 1.0
Autenticación de proveedores API	Preventivo	APIM Externo	mTLS + OAuth 2.0 + PKCE	FAPI 1.0 Advanced
Autenticación inter-servicio	Preventivo	APIM Externo/Istio	mTLS automático (Istio) + JWT servicio	Zero Trust
Autenticación inter-servicio	Preventivo	Microservicio	RBAC (empleados) / ABAC + scopes (clientes)	OAuth 2.0 scopes
Protección DDoS	Preventivo	Perímetro	Azure DDoS Protection Standard	ISO 27001
Web Application Firewall	Preventivo / Detectivo	APIM Externo / Front Door	Azure WAF (OWASP Top 10 ruleset)	OWASP, PCI DSS 6.6
Rate Limiting	Preventivo	APIM Externo	Azure APIM policies (por client_id, IP)	FAPI, PCI DSS
Cifrado en tránsito	Preventivo	Todas las capas	TLS 1.3 obligatorio; TLS 1.0/1.1 deshabilitado	PCI DSS 4.0, FIPS 140-2
Cifrado en reposo	Preventivo	Bases de datos / Discos	AES-256 (Azure Disk Encryption + TDE)	PCI DSS 3.4, ISO 27001
Cifrado campo-nivel PII	Preventivo	Customer Service	AES-256-GCM por campo (IBAN, documento)	GDPR / LOPD
Gestión de secretos	Preventivo	Toda la plataforma	Azure Key Vault + CSI Driver (AKS)	ISO 27001 A.10
Escaneo de vulnerabilidades	Preventivo	CI/CD Pipeline	Trivy (containers) + SAST (SonarQube)	DevSecOps
Microsegmentación de red	Preventivo	Red interna	Azure NSG + Private Endpoints + VNet	Zero Trust, ISO 27001
Auditoría de acceso a PII	Detectivo / Correctivo	Microservicios + Log Analytics	Structured logs obligatorios por operación PII	GDPR Art. 30, LOPD
Detección de fraude en tiempo real	Detectivo	Fraud Detection Service	Modelo ML + Kafka (<50ms p99)	BIAN Fraud Detection SD
Rotación automática de certificados	Preventivo	Infraestructura	cert-manager (Kubernetes) + Key Vault	PCI DSS 4.0
Backup y recuperación	Correctivo	Bases de datos	Azure Backup + Geo-Replication	ISO 22301 (BCP)
Pod Security Policies	Preventivo	AKS (contenedores)	OPA Gatekeeper + Non-root containers	CIS Kubernetes Benchmark